

**ROLLING STOCK WORKSHOP
NORTHERN RAILWAY, CHARBAGH
LUCKNOW**



A SUMMER TRAINING REPORT ON

SOFTWARE DEVELOPMENT FOR DATA KEEPING OF STC

(Joining, Feedback, Training and Exam & Marks etc.)



SUBMITTED BY

**RISHABH GUPTA (TEAM LEAD)
AWANISH YADAV
PRATYUSH KUMAR SINGH
PRIYANSHU RAJ
SOMESH PRATAP SINGH
SHIVAM YADAV
VENKATESH
PALLAVI TRIPATHI
SHIVANSH KUMAR**

SUBMITTED TO

**MR. MANVENDRA SINGH
(Chief Instructor)**

DECLARATION

We hereby declare that this Industrial Training Report entitled “Trainee Management Software for Development of Data Management of STC & WTC (including Joining, Feeding, Training, Examination & Marks, etc.)” at Locomotive Workshop, Northern Railway, Charbagh, Lucknow is the result of our own work and effort. To the best of our knowledge and belief, it contains no material previously published or written by any other person, nor material which has been accepted for the award of any other degree or diploma of the University or any other institute of higher education, except where due acknowledgement has been made in the text.

Rishabh Gupta (Team Lead)

Date:

Awanish Yadav

Date:

Pratyush Kumar Singh

Date:

Priyanshu Raj

Date:

Somesh Pratap Singh

Date:

Shivam Yadav

Date:

Venkatesh

Date:

Pallavi Tripathi

Date:

Shivansh Kumar

Date:

ABSTRACT

This report outlines the industrial training project undertaken at Northern Railway, Charbagh, Lucknow, centered on the development of a Trainee Management Software for the Supervisor's Training Centre (STC). The objective of the project was to design a digital solution to streamline the management of trainee-related processes such as joining records, daily feeding, training schedules, examinations, and marks entry.

The training was conducted in coordination with both the Supervisor's Training Centre (STC) and Workshop Training Centre (WTC), whose support and guidance were essential in understanding the operational requirements and challenges. The project followed a structured development process, starting from requirement gathering to system design, implementation, testing, and documentation.

This software aims to replace traditional manual processes with an efficient, accurate, and centralized data management system. It is expected to enhance administrative productivity, ensure data integrity, and improve the overall management of training operations within STC and WTC.

Through this training, we gained valuable exposure to real-world software development practices and strengthened our ability to work collaboratively in a professional environment.

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to Northern Railway, Charbagh, Lucknow, for providing us the opportunity to undertake our industrial training at their prestigious organization. This training experience has been instrumental in enhancing our practical understanding of software development in a real-world environment.

We are especially thankful to the Supervisor's Training Centre (STC) and Workshop Training Centre (WTC) for their continuous support and collaboration during the project. Our heartfelt thanks to our project mentor [**Manavendra Singh, Naveen Jaiswal and Alok Dixit**] for their expert guidance, valuable feedback, and encouragement throughout the development of the Trainee Management Software for Data Management of STC.

We are also grateful to our faculty members and department at University of Lucknow for facilitating this training and supporting us academically and technically during the project.

Finally, we would like to acknowledge the efforts and cooperation of every team member whose dedication and teamwork made the successful completion of this training and project possible.

TABLE OF CONTENTS

	Page No.
Declaration	ii
Abstract	iii
Acknowledgement	iv
Table of Contents	v
List of Images	ix
List of Tables	x
CHAPTER 1: INTRODUCTION	1-7
1.1 GENERAL	1
1.2 PROJECT OVERVIEW	1
1.3 FEATURES OF THE PLATFORM	2
1.4 TECHNOLOGIES USED	4
1.5 APPLICATION ARCHITECTURE	5
1.5.1 Model (Data Layer)	5
1.5.2 View (User Interface Layer)	6
1.5.3 Controller (Application Logic Layer)	6
1.5.4 Hosting and Execution	6
1.5.5 Version Control	6
1.6 SUMMARY	7
CHAPTER 2: DESIGN AND SOFTWARE IMPLEMENTATION	8-14
2.1 INTRODUCTION	8
2.2 MVC ARCHITECTURE	8
2.2.2 Model: Data Layer	8
2.2.2 View: User Interface Layer	9
2.2.3 Controller: Logic Layer	9
2.2.4 Modularity and Scalability	10
2.3 ENTITY RELATION / SYSTEM DESIGN	10
2.3.1 Core Entities and Data Flow	10
2.3.2 Workflow Overview	10
2.3.3 System Design Principles	11
2.3.4 Why Relational Design (SQLite3)?	11
2.4 FRONTEND IMPLEMENTATION	11
2.5 BACKEND IMPLEMENTATION	12
2.6 DATABASE STRUCTURE	12
2.7 SUMMARY	14

CHAPTER 3: BACKEND CODE ARCHITECTURE AND IMPLEMENTATION	15-89
3.1 OVERVIEW	15
3.2 SERVER ENTRY POINT	15
3.2.1 server.js	15
3.3 CONFIGURATION LAYER	18
3.3.1 Database Configuration	18
3.3.2 Backend Configuration	20
3.4 MIDDLEWARE LAYER	22
3.4.1 Authentication Middleware	22
3.4.2 Image Upload Middleware	24
3.5 UTILITIES LAYER	25
3.5.1 Session Management	25
3.5.2 Unique Ticket Number Generator	27
3.6 CONTROLLERS LAYER	30
3.6.1 Authentication Controller	30
3.6.2 Database Export to Excel	32
3.6.3 Line Training Controller	36
3.6.4 Monthly Attendance Controller	43
3.6.5 Candidate Management Controllers	47
3.6.6 Course Score Controllers	49
3.7 ROUTES LAYER	54
3.7.1 Authentication Routes	54
3.7.2 Export Routes	55
3.7.2 Line Training Routes	56
3.7.3 Monthly Attendance Routes	57
3.7.4 Candidate Management Routes	58
3.7.5 Course Score Management Routes	60
3.8 MODEL LAYER	62
3.8.1 User Model	62
3.8.2 Line Training Model	63
3.8.3 Monthly Attendance Model	70
3.8.4 Candidate Data Models	82
3.8.5 Course Score Models	84
3.9 COMPLETE AUTHENTICATION FLOW	87
3.9.1 Authentication Architecture Overview	87
3.9.2 Authentication Flow Diagram	87
3.9.3 Training Module Architecture	87
3.10 BACKEND ARCHITECTURE SUMMARY	88
3.10.1 Technology Stack	88
3.10.2 Request Flow	88

3.10.3 Key Design Patterns	88
3.10.4 Security Features	89
3.10.5 File Structure Benefits	89
3.10.6 Data Flow Example	89
3.10.7 Error Handling Strategy	89
CHAPTER 4: FRONTEND CODE ARCHITECTURE AND IMPLEMENTATION	90-107
4.1 INTRODUCTION	90
4.2 PROJECT STRUCTURE	91
4.2.1 Top-Level Files and Folders	91
4.2.2 Source Directory	92
4.3 STATE MANAGEMENT AND ROUTING	93
4.4 MARKSHEET IMPLEMENTATION	94
4.4.1 Overview	94
4.4.2 Component Structure	94
4.4.3 Data Flow	95
4.4.4 UI/UX Considerations	96
4.5 CERTIFICATE IMPLEMENTATION	98
4.5.1 Overview	98
4.5.2 Component Structure	98
4.5.3 Certificate Generation Workflow	101
4.5.4. UI/UX Considerations	102
4.6 LETTER IMPLEMENTATION	103
4.6.1 Overview	103
4.6.2 Component Structure	103
4.6.3 UI/UX Considerations	104
4.7 BEST PRACTICES, CHALLENGES, AND CONCLUSION	105
4.7.1 Best Practices and Patterns	106
4.7.2 Challenges and Solutions	106
4.7.3 Conclusion	107
CHAPTER 5: TESTING RESULT	108-127
5.1 INTRODUCTION	108
5.1.1 Testing Methodology	108
5.1.2 Module-Wise Test Coverage	109
5.2 ADD & MANAGE CANDIDATE FOR STC, WTC, NON-RAILWAY	109
5.2.1 STC Form	110
5.2.2 WTC Form	111
5.2.3 Non-Railway Form	113
5.2.4 Image Handling	114
5.2.5 Filtration	115

5.3 STC MANAGEMENT SYSTEM	116
5.3.1 Feed Marks	116
5.3.2 Trainee Profile	117
5.3.3 Line Training	119
5.3.4 Marksheet Generation	120
5.4 WTC MANAGEMENT	121
5.4.1 Trainee Profile	121
5.4.2 Attendance System	123
5.4.3 Certificate Generation	124
5.4.4 Letter Generation	125
5.5 DISCUSSIONS	126
5.6 CONCLUSION	127
CHAPTER 6: CONCLUSIONS AND SCOPE FOR IMPROVEMENT	128-132
6.1 INTRODUCTION	128
6.2 CONCLUSIONS	128
6.2.1 Objective Achievement	128
6.2.2 Technical Implementation Success	129
6.2.3 Functional Module Delivery	129
6.2.4 Performance Impact	129
6.2.5 System Reliability	129
6.2.6 Performance and Efficiency Gains	129
6.3 FUTURE SCOPE	130
6.3.1 Online Deployment and Accessibility	130
6.3.2 Database Enhancement and Scalability	130
6.3.3 Security Infrastructure and Access Control	130
6.3.4 Advanced User Management System	131
6.3.5 Enhanced Reporting and Analytics	131
6.4 CONCLUSION	131
APPENDIX	
CONTRIBUTORS	

LIST OF IMAGES

	Page No.
Fig. 2.1 Use Case Flow Diagram	13
Fig. 4.1 Login Page	91
Fig. 4.2 Marksheet Preview	98
Fig. 4.3 STC Certificate Preview	102
Fig. 4.4 WTC Certificate Preview	103
Fig. 4.5 Letter Preview	105
Fig. 5.1 Manage Candidate Page	110
Fig. 5.2 STC Registration	111
Fig. 5.3 WTC Registration	112
Fig. 5.4 Non-Railway Registration	114
Fig. 5.5 Feed Marks Page	117
Fig. 5.6 Candidate Profile	119
Fig. 5.7 Line Training Interface	120
Fig. 5.8 Marksheet Generation Interface	121
Fig. 5.9 Trainee Profile Page	122
Fig. 5.10 Trainee Attendance Summary	124
Fig. 5.11 Certificate Generation Interface	125
Fig. 5.12 Letter Generation Interface	126

LIST OF TABLES

	Page No.
Table 2.1 Database Schema	12-13
Table 5.1 Project Module Testing	109

CHAPTER 1

INTRODUCTION

1.1 GENERAL

The Trainee Management Software for Data Management of STC represents a modern solution aimed at streamlining and digitizing the trainee-related administrative processes within the Supervisor's Training Centre (STC) and Workshop Training Centre (WTC) at Northern Railway, Charbagh, Lucknow. These centers play a crucial role in the training and upskilling of technical staff and supervisors, ensuring that the workforce is competent, disciplined, and well-equipped to manage railway operations efficiently.

Traditionally, the management of trainee records—such as joining details, feeding records, training schedules, assessments, and marks—has relied on manual processes. These methods are not only time-consuming but also prone to human error, data loss, duplication, and administrative inefficiencies. The lack of a centralized digital system often leads to difficulties in data retrieval, tracking trainee progress, and generating analytical reports.

In today's rapidly evolving railway ecosystem, where efficiency, accuracy, and real-time data access are paramount, there is a growing need to adopt smart digital systems in training and administrative operations. The implementation of such a software not only enhances operational efficiency but also contributes to improved decision-making, better resource management, and transparency in trainee evaluation. By transitioning from manual to digital, the system aligns with the broader vision of modernization and digital transformation within Indian Railways.

1.2 PROJECT OBJECTIVE

The primary objective of the Trainee Management Software for Data Management of STC is to develop a comprehensive and user-friendly digital system that automates and simplifies the management of trainee-related information within the Supervisor's Training

Centre (STC) and Workshop Training Centre (WTC) of Northern Railway, Charbagh, Lucknow.

The specific objectives of this project are as follows:

- **To digitize trainee records** such as joining details, personal information, feeding logs, training schedules, examination details, and marks in a structured and centralized manner.
- **To eliminate manual paperwork and reduce human errors** by providing a reliable, fast, and accurate system for data entry, retrieval, and reporting.
- **To streamline the training workflow**, from registration to evaluation, ensuring seamless coordination among trainees, trainers, and administrative staff.
- **To enable real-time access** to trainee data, performance reports, and attendance summaries through a well-organized dashboard for administrators and supervisors.
- **To improve data security and integrity** by implementing access control features, ensuring only authorized personnel can view or edit sensitive trainee information.
- **To enhance overall efficiency and transparency** in the functioning of STC/WTC by automating repetitive tasks and reducing administrative burden.
- **To support data-driven decision-making** by generating performance analytics and reports that help trainers and authorities evaluate training outcomes and identify areas of improvement.

By achieving these objectives, the software aims to become a valuable digital asset in the ongoing modernization and digital transformation of training operations within Indian Railways.

1.3 FEATURES OF THE PLATFORM

The Trainee Management Software for Data Management of STC has been designed with a focus on usability, functionality, and efficiency. It includes a range of features to support the smooth handling of various trainee-related tasks by the Supervisor's Training Centre (STC) and Workshop Training Centre (WTC). These features ensure that administrative processes are streamlined, organized, and digitally accessible.

The key features of the platform include:

1. Centralized Trainee Database

- Maintains all trainee information including personal details, joining date, department, and training type.

- Easily accessible and editable by authorized personnel.

2. Feeding and Attendance Management

- Daily feeding data and attendance records are maintained digitally.
- Quick tracking of trainee presence and consumption logs.

3. Training Schedule Management

- Allows scheduling of training sessions, practical workshops, theory classes, and assessments.
- Trainees and instructors can view daily/weekly training schedules in a structured format.

4. Examination and Marks Entry

- Records internal/external exam details and trainee performance.
- Enables smooth entry and modification of marks by authorized trainers or evaluators.

5. Role-Based Access Control

- Separate login credentials for administrators, trainers, and supervisors.
- Ensures data privacy and prevents unauthorized access or modifications.

6. Report Generation

- Automatically generates individual and batch-wise reports for attendance, marks, and overall performance.
- Reports can be exported and printed as per administrative requirements.

7. Search and Filter Functionality

- Allows quick search and filtering of records based on name, ID, batch, date, or status.
- Saves time and improves efficiency in handling bulk data.

8. User-Friendly Interface

- Simple and intuitive UI design for ease of use by non-technical staff.
- Minimal training required to operate the system.

9. Data Backup and Recovery

- Regular data backup features ensure that trainee records are safely stored.
- In case of system failure, records can be restored without loss.

10. Scalability and Flexibility

- The platform can be scaled to accommodate additional features or expanded usage across departments or locations.
- Modular architecture allows easy future upgrades.

1.4 TECHNOLOGIES USED

The development of the Trainee Management Software for Data Management of STC leveraged modern and efficient technologies to ensure a responsive, scalable, and maintainable application. The selected tech stack was chosen based on ease of integration, speed of development, and suitability for a local deployment environment.

1. Frontend Technologies

- **React.js:** A JavaScript library used to build a dynamic and component-based user interface.
- **Tailwind CSS:** A utility-first CSS framework used for styling and designing a clean, responsive UI.
- **HTML2PDF:** Used to generate downloadable PDF reports of trainee records and summaries.
- **React Icons:** A library that provides popular icon sets for UI enhancement.
- **Manual Validation:** Custom validation logic implemented for form inputs to ensure data accuracy before submission.

2. Backend Technologies

- **Node.js:** JavaScript runtime environment used to build the server-side logic of the application.
- **Express.js:** A lightweight Node.js framework used for routing, handling requests, and connecting to the database.
- **Nodemon:** A development tool used to monitor changes in source files and automatically restart the server for efficient testing and debugging.
- **Proxy:** Configured for seamless connection between frontend and backend during development.

3. Database

- **SQLite3:** A lightweight, file-based relational database used for storing trainee information, attendance records, marks, and training schedules. Ideal for local and embedded systems due to its minimal configuration requirements.

4. Server and Hosting

- **Localhost:** The application was hosted and tested on a local development server for demonstration purposes during the industrial training.

5. Version Control

- **Git & GitHub:** Used for version control, code management, and team collaboration throughout the development lifecycle.

1.5 APPLICATION ARCHITECTURE

The Trainee Management Software for Data Management of STC is structured around the Model-View-Controller (MVC) architectural pattern, which separates the application into three main components to ensure modularity, clarity, and maintainability. This layered approach helps streamline development, debugging, and future enhancements.

1.5.1 Model (Data Layer)

The Model layer manages all data-related logic using SQLite3 — a lightweight, embedded relational database. It is responsible for storing and retrieving all essential information such as:

- Trainee profiles (name, batch, department, etc.)
- Attendance and feeding data
- Training schedules
- Examination details and marks

This layer handles data persistence and ensures that all information is consistently structured and retrievable across the system.

1.5.2 View (User Interface Layer)

The View is developed using React.js and styled with Tailwind CSS to offer a clean, responsive, and user-friendly interface. Key features include:

- Dynamic form handling for trainee data entry and updates
- Manual validation to ensure data accuracy
- Table views for visualizing trainee information and performance
- Report generation using HTML2PDF
- Icon-based UI enhancements using React Icons

This layer interacts directly with the backend functions via tightly coupled routes and local communication, eliminating the need for API endpoints.

1.5.3 Controller (Application Logic Layer)

The Controller is built with Node.js and Express.js, responsible for coordinating actions between the frontend and the database. It processes user inputs, applies business logic (such as validation and conditional actions), and communicates with the model to fetch or update records.

Unlike traditional web applications that rely on external APIs, your system uses direct internal routing and logic, facilitated via proxy configuration to allow seamless communication between the React frontend and Express backend during development.

1.5.4 Hosting and Execution

The entire application is hosted locally using localhost for development and demonstration. The application can be accessed by any device on the network using the server's IP address. Tools like Nodemon are used to auto-reload the backend server on code changes, enabling faster testing and debugging.

1.5.5 Version Control

All source code and updates were managed using Git and GitHub, ensuring version tracking, collaboration, and backup throughout the project lifecycle.

1.6 SUMMARY

This chapter introduced the Trainee Management Software developed for the STC and WTC at Northern Railway, Charbagh, Lucknow. It highlighted the need for digitizing trainee data management, replacing manual processes with a more efficient digital system. The objectives, key features, and technologies used were outlined. The application architecture based on the MVC pattern was also discussed, ensuring modularity and ease of maintenance. This chapter sets the foundation for understanding the system's purpose and structure.

CHAPTER 2

DESIGN AND SOFTWARE IMPLEMENTATION

2.1 INTRODUCTION

This chapter presents the design and software implementation of the Trainee Management Software. It explains how the system was structured and built to meet the operational needs of trainee data management.

The design process included planning the user interface, defining the database schema, and outlining the logic of individual modules. Technologies such as React.js for the frontend, Node.js and Express for backend development, and SQLite3 for data handling were used during the implementation phase.

This chapter will detail the visual layout, data structures, and internal workflows that bring the system together as a fully functional application.

2.2 MVC ARCHITECTURE

The Model-View-Controller (MVC) architectural pattern provides a well-structured framework for developing the Trainee Management Software, promoting clear separation of concerns between the data layer, user interface, and business logic. This separation enhances maintainability, scalability, and ease of future modifications.

2.2.2 Model: Data Layer

The Model represents the data-centric layer of the system. It encapsulates all trainee-related records and supports efficient data retrieval and manipulation. Key responsibilities include:

- Managing structured data such as trainee details, batch allocation, feeding records, attendance logs, exam scores, and training schedules.
- Ensuring data integrity and consistency through interactions with the database.
- Supporting structured querying for generating reports and visual summaries.

This centralized data layer provides a reliable foundation for system operations and analytics.

2.2.2 View: User Interface Layer

The View is responsible for presenting data and interacting with users. Built using React.js and styled with Tailwind CSS, it includes intuitive, role-based interfaces that enable smooth interaction with the system. Key features include:

- Trainee registration forms, attendance displays, and marks entry panels.
- Dashboards for trainers/administrators to view and manage training workflows.
- Manual client-side validation to ensure data accuracy before submission.

This layer is designed to be modular and adaptable, allowing interface updates without affecting backend logic.

2.2.3 Controller: Logic Layer

The Controller, built with Node.js and Express.js, serves as the bridge between the View and the Model. It processes inputs, applies business rules, and coordinates system behavior. Its functions include:

- Handling route logic and form submissions.
- Performing validation and control checks before accessing or modifying the database.
- Managing communication between frontend and backend using internal routes configured via proxy.

APIs are used to enable structured and secure communication between the system components, allowing the frontend to interact with the backend through clearly defined endpoints.

2.2.4 Modularity and Scalability

The adoption of the MVC pattern ensures loose coupling and separation of concerns, making the application modular, easy to debug, and scalable. New features—such as additional reporting modules, user roles, or attendance analytics—can be added with minimal impact on existing components.

2.3 ENTITY RELATION / SYSTEM DESIGN

The system design of the Trainee Management Software is centered around structured and relational data management. It focuses on efficiently capturing and organizing trainee information, attendance records, feeding logs, training schedules, and exam results. The data model follows a relational structure, implemented through SQLite3, and is designed to ensure integrity, scalability, and ease of access for reporting and analysis.

2.3.1 Core Entities and Data Flow

The application's core revolves around managing a trainee's journey from joining to training, feeding, examination, and certification. The key entities in the system include:

- **Trainee:** Central to the system, storing personal and academic details such as name, batch, department, and contact information.
- **Attendance Record:** Logs daily attendance per trainee, linked via Trainee ID, with date and presence status.
- **Record Feeding:** Refers to the entry of trainee records into the system, ensuring each trainee's progress is accurately recorded and stored.
- **Training Schedule:** Manages session details like session title, date, and batch.
- **Exam Result:** Stores trainee exam information, including subjects, marks, and generating marksheet.

2.3.2 Workflow Overview

- **Data Entry:** Admins or coordinators add trainee data, training sessions, and course schedules via user-friendly forms.
- **Daily Usage:** Attendance is updated on a daily basis.
- **Tracking & Updates:** Trainers can view and modify records based on trainee progress.

- **Reporting:** All data can be exported in Excel Spreadsheet format for reporting and documentation.

2.3.3 System Design Principles

- **Centralized Data Control:** The use of a single relational database ensures consistency and data reliability.
- **Structured Schema:** Tables are designed to maintain one-to-many relationships between trainees and their respective records (attendance, exams, feeding, etc.).
- **Separation of Concerns:** Aligned with the MVC architecture, data is isolated in the model layer for clean access and management.
- **Simplicity and Modularity:** The system is kept modular to allow for easy extension (e.g., adding new features like certification or feedback tracking).

2.3.4 Why Relational Design (SQLite3)?

Unlike NoSQL systems used in dynamic, unstructured use cases, the structured nature of trainee records suits a relational model. With a consistent schema (trainee -> schedule -> result/attendance), SQLite3 provides:

- Fast lookups
- Simplified queries
- Lightweight storage for local deployment
- Easier report generation and backups

2.4 FRONTEND IMPLEMENTATION

The NR Trainee Management System's frontend presents a streamlined interface optimized for administrative tasks throughout module. Its layout includes:

- **Navigation Bar:** Located at the top, it allows access to candidate management features and multiple modules, emphasizing usability through clear links.
- **Dashboard Cards:**
 - Trainee Profile – Centralized access to trainee details.
 - Attendance System – Interface likely captures daily presence data.
 - Certificate Generation – Enables automated issuance of training certificates.
 - Letter Generation – Supports administrative documentation processes.

- **User Interaction Enhancements:**
 - Display of system status (“Active”) and current date fosters real-time context awareness.
 - Visible admin login and logout options suggest role-based access controls.
- **Design Style:** Visually clean, icon-driven cards with short descriptions support rapid task execution with minimal clutter.

2.5 BACKEND IMPLEMENTATION

The backend of the NR Trainee Management System is developed using Node.js with Express.js, offering a lightweight and efficient server-side solution. Key elements include:

- **Server Configuration:**
 - Express routes define structured endpoints for modules like trainee management, attendance logging, certification, and document handling.
 - nodemon is integrated to support hot reloading, ensuring developers can make live updates during development.
- **Modular Structure:**
 - Routes, controllers, and database logic are separated for maintainability.
 - Middleware handles request parsing, CORS, error logging, and proxy routing.
- **Proxy Setup:**
 - A reverse proxy (e.g., using http-proxy-middleware) is implemented to facilitate secure communication between frontend and backend services during development.

2.6 DATABASE STRUCTURE

The application uses **SQLite3**, a file-based lightweight relational database ideal for fast read/write operations with minimal setup. Sample schema:

Table 3.1 Database Schema

Table Name	Fields	Functionality
stc_candidates	id, ticket_no, name, designation, unit, dob, sex, category, pwd, type_of_disability, contact details, address details, course details, session1start-session4end, created_at	Stores candidate details under STC module

Table Name	Fields	Functionality
wtc_candidates	id, ticket_no, name, designation, unit, dob, sex, category, pwd, type_of_disability, contact details, address details, course details, course_coordinator, created_at	Stores candidate details under WTC module
Users	id, role, password, created_at	Manages admin, master, and operator access
attendances	id, candidate_id, ticket_no, total_classes, classes_attended, created_at, updated_at	Tracks total and attended classes for a candidate
line_training_candidates	id, ticket_no, name, designation, activity_centre, start_date, end_date, remark, created_at	Manages line training records and progress

Relationships are defined via foreign keys (trainee_id) to enable structured joins and data retrieval.

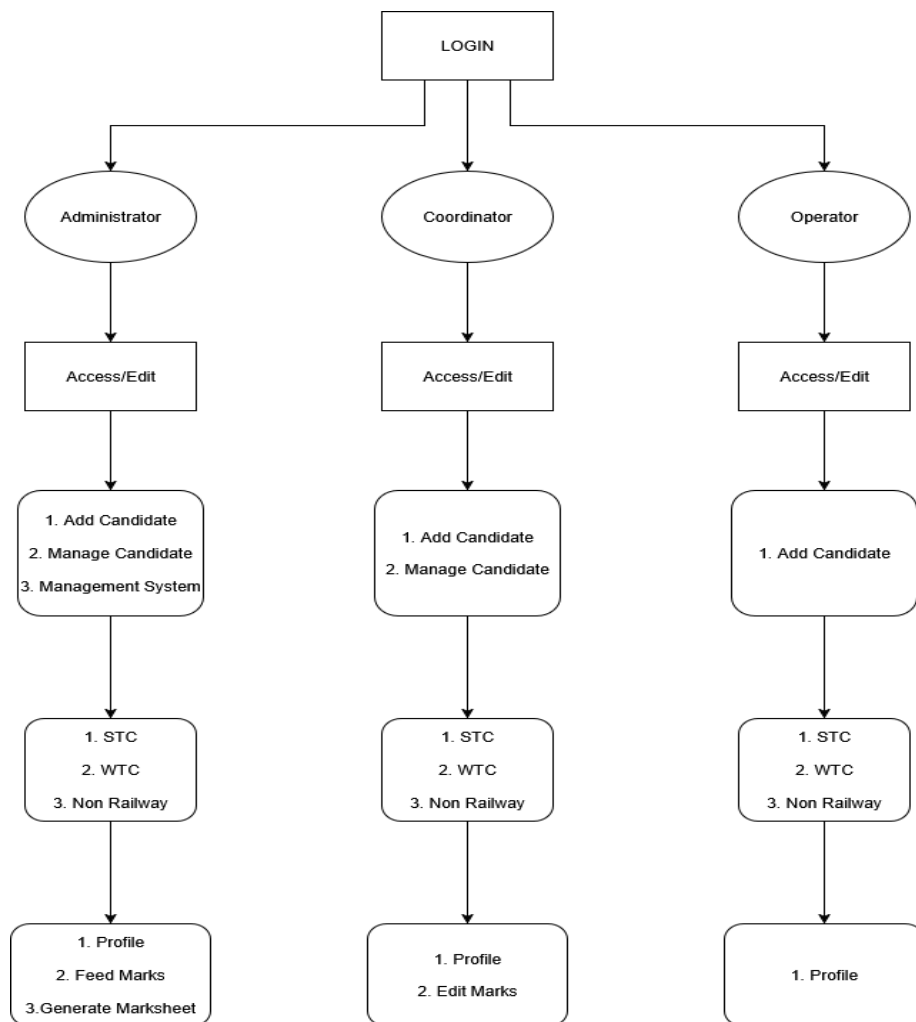


Fig. 2.1 Use Case Flow Diagram

2.7 SUMMARY

This chapter summarized the core design and development aspects of the Training Management System. It highlighted the use of the MVC architecture for clean code separation, a responsive frontend for user interaction, a logic-driven backend for data handling, and a structured database for efficient storage. Together, these components formed a scalable and reliable solution tailored for managing training processes in STC, WTC, and Non-Railway domains

CHAPTER 3

BACKEND CODE ARCHITECTURE AND IMPLEMENTATION

3.1 OVERVIEW

The STVT backend is built using Node.js with Express.js framework and SQLite3 as the database. It follows a modular architecture with clear separation of concerns, implementing the MVC (Model-View-Controller) pattern for maintainable and scalable code.

3.2 SERVER ENTRY POINT

3.2.1 server.js

The main entry point that initializes the Express application, configures middleware, and starts the server.

Key Responsibilities:

- Application initialization
- Middleware registration
- Route mounting
- Database connection
- Server startup

```
const express = require("express");
const cors = require("cors");
const path = require("path");
const { initializeDatabase } = require("../config/db");
const fs = require("fs");
const os = require("os");

// Import all routes
const authRoutes = require("../routes/authRoutes");
```

```

const stcRoutes = require("./routes/stcRoutes");
const wtcRoutes = require("./routes/wtcRoutes");
const nonRailwayRoutes = require("./routes/nonRailwayRoutes");
const monthlyAttendanceRoutes =
require("./routes/monthlyAttendanceRoutes");
const lineTrainingRoutes = require("./routes/lineTrainingRoutes");
const exportRoutes = require("./routes/exportRoutes");

// Course-specific routes (12 total)
const mjiCwRoutes = require("./routes/mjicwRoutes");
const mjidRoutes = require("./routes/mjidRoutes");
const mjiwRoutes = require("./routes/mjiwRoutes");
const mjpCwRoutes = require("./routes/mjpcwRoutes");
const mjpdRoutes = require("./routes/mjpdRoutes");
const mjpwRoutes = require("./routes/mjpwRoute");
const mjrCwRoutes = require("./routes/mjrcwRoute");
const mjrdRoutes = require("./routes/mjrdRoute");
const mjrWRoutes = require("./routes/mjrWRoutes");
const mseCwRoutes = require("./routes/msecwRoutes");
const msedRoutes = require("./routes/msedRoutes");
const msewRoutes = require("./routes/msewRoutes");

const app = express();

// Initialize the database
initializeDatabase();

// Middleware configuration
app.use(cors({ origin: "*" }));
app.use(express.json());
app.use(express.urlencoded({ extended: true }));

// Serve uploaded images
app.use("/uploads", express.static(path.join(__dirname, "uploads")));

// API Routes
app.use("/api/auth", authRoutes);
app.use("/api/stc", stcRoutes);
app.use("/api/wtc", wtcRoutes);
app.use("/api/nonrailway", nonRailwayRoutes);
app.use("/api/monthly-attendance", monthlyAttendanceRoutes);
app.use("/api/export-database", exportRoutes);
app.use("/api/line-trainings", lineTrainingRoutes);

// 12 Course-specific routes
app.use("/api/mse-c&w", mseCwRoutes);
app.use("/api/mse-d", msedRoutes);
app.use("/api/mse-w", msewRoutes);
app.use("/api/mjr-c&w", mjrCwRoutes);
app.use("/api/mjr-d", mjrdRoutes);
app.use("/api/mjr-w", mjrWRoutes);
app.use("/api/mji-c&w", mjiCwRoutes);
app.use("/api/mji-d", mjidRoutes);
app.use("/api/mji-w", mjiwRoutes);
app.use("/api/mjp-c&w", mjpCwRoutes);
app.use("/api/mjp-d", mjpdRoutes);
app.use("/api/mjp-w", mjpwRoutes);

```

```

// Health check endpoint
app.get("/api/health", (req, res) => {
  res.json({
    status: "OK",
    message: "Trainee Management System API is running",
    timestamp: new Date().toISOString(),
  });
});

// 404 handler
app.use("*", (req, res) => {
  res.status(404).json({
    success: false,
    message: "Route not found",
  });
});

// Global error handler
app.use((err, req, res, next) => {
  console.error("Global error:", err);
  res.status(500).json({
    success: false,
    message: "Internal server error",
    error: process.env.NODE_ENV === "development" ? err.message :
undefined,
  });
});

// Network utilities
function getLocalIP() {
  const interfaces = os.networkInterfaces();
  for (const name of Object.keys(interfaces)) {
    for (const iface of interfaces[name]) {
      if (iface.family === "IPv4" && !iface.internal) {
        return iface.address;
      }
    }
  }
  return "localhost";
}

function updateFrontendEnv(ip) {
  const envPath = path.join(__dirname, "../frontend/.env");
  let envContent = fs.readFileSync(envPath, "utf-8");
  envContent = envContent.replace(
    /VITE_BACKEND_IP=.*\/g,
    `VITE_BACKEND_IP=${ip}`
  );
  fs.writeFileSync(envPath, envContent, "utf-8");
}

// Server startup
const PORT = process.env.PORT || 5000;
const localIP = getLocalIP();
updateFrontendEnv(localIP);

```

```
app.listen(PORT, "0.0.0.0", () => {
  console.log(`🚀 Server running on port ${PORT}`);
  console.log(`Local: http://localhost:${PORT}/api`);
  console.log(`Network: http://${localIP}:${PORT}/api`);
});
```

Key Features:

- **Express Application Setup:** Initializes Express server with necessary middleware
- **Database Initialization:** Calls initializeDatabase() to set up SQLite database
- **CORS Configuration:** Allows cross-origin requests from any origin
- **Static File Serving:** Serves uploaded files from /uploads directory
- **Comprehensive Route Registration:** Includes 12 course-specific routes plus core functionality
- **Error Handling:** Global error handler with development/production modes
- **Network Configuration:** Automatically detects local IP and updates frontend environment
- **Health Check:** Provides API status endpoint for monitoring

3.3 CONFIGURATION LAYER

3.3.1 Database Configuration

Handles SQLite database connection and initialization.

File: config/db.js

```
const sqlite3 = require('sqlite3').verbose();
const { join } = require('path');
const fs = require('fs');

// Directory structure setup
const dbDir = join(__dirname, '..', 'db');
const uploadsDir = join(__dirname, '..', 'uploads');
const tempDir = join(uploadsDir, 'temp');

// Create directories if they don't exist
[dbDir, uploadsDir, tempDir].forEach(dir => {
  if (!fs.existsSync(dir)) {
    fs.mkdirSync(dir, { recursive: true });
  }
});

// Set up the full path for the SQLite database file
const dbPath = join(dbDir, 'db.sqlite');

// Create a new SQLite database instance
```



```

const db = new sqlite3.Database(dbPath)

// Function to set up the database schema
const initializeDatabase = () => {
  // db.serialize() ensures sequential execution of database operations
  db.serialize(() => {
    // Create users table with role-based authentication
    db.run(`
      CREATE TABLE IF NOT EXISTS users (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        role TEXT UNIQUE NOT NULL CHECK (role IN ('admin',
'master', 'operator')),
        password TEXT NOT NULL,
        created_at DATETIME DEFAULT CURRENT_TIMESTAMP
      )
    `, (err) => {
      if (err) {
        console.error('Error creating users table:',
err.message);
      } else {
        console.log('Users table checked/created successfully.
Initialized with default passwords');
      }
    });

    // Insert default users with role-based access
    db.run(`
      INSERT OR IGNORE INTO users (role, password)
      VALUES
        ('admin', '1234'),
        ('master', '5678'),
        ('operator', '9012')
    `);
  });
};

module.exports = { db, initializeDatabase, dbPath };

```

Key Features:

- **SQLite Database:** Uses SQLite for local data storage instead of MongoDB
- **Directory Management:** Automatically creates necessary directories (db, uploads, temp)
- **Database Initialization:** Creates users table with role-based authentication
- **Default Users:** Initializes system with three default user roles
- **Error Handling:** Comprehensive error handling for database operations
- **Sequential Operations:** Uses db.serialize() to ensure proper operation order

Database Schema:

- **Users Table:** Stores user credentials with role-based access control
 - id: Primary key (auto-increment)
 - role: User role (admin, master, operator) with uniqueness constraint
 - password: User password (stored as plain text - should be hashed in production)
 - created_at: Timestamp of user creation

3.3.2 Backend Configuration

The config.js file serves as a central configuration hub for the backend server. It defines essential server settings such as the port number, CORS origin, and environment mode, all of which can be customized via environment variables. The file also specifies the session expiry duration, ensuring user sessions are managed securely. A significant feature of this configuration is the rolePermissions object, which outlines the permissions associated with different user roles (admin, master, and operator). These permissions govern access to various features and functionalities within the application, supporting a robust role-based access control system. All these configurations are exported for use across the backend codebase, promoting consistency and maintainability.

File: config/config.js

```
// Server configuration
const PORT = process.env.PORT || 5000;
const CORS_ORIGIN = process.env.CORS_ORIGIN || 'http://localhost:5173';
const NODE_ENV = process.env.NODE_ENV || 'development';

// Session configuration
const SESSION_EXPIRY = 24 * 60 * 60 * 1000; // 24 hours

// Role permissions
const rolePermissions = {
  admin: [
    // Candidate Management
    "add-candidate",
    "manage-candidates",

    // STC Features
    "stc-feed-marks",
    "stc-trainee-profile",
    "stc-line-training",
    "stc-marksheet",
    "stc-management",

    // WTC Features
    "wtc-feed-marks",
```

```

        "wtc-trainee-profile",
        "wtc-attendance",
        "wtc-letter",
        "wtc-certificate",
        "wtc-management",

        // Non-Railway Features
        "non-railway-management",

        // Forms
        "stc-form",
        "wtc-form",
        "non-railway-form",

        // Generic permissions - mapped to UI elements
        "feed-marks",      // Generic menu item
        "marksheets",      // Generic menu item
        "letters",          // Generic menu item
        "trainee-profile" // Generic menu item
    ],
    master: [
        // Candidate Management
        "add-candidate",
        "manage-candidates",

        // STC Features
        "stc-feed-marks",
        "stc-trainee-profile",
        "stc-line-training",
        "stc-management",

        // WTC Features
        "wtc-trainee-profile",
        "wtc-attendance",
        "wtc-management",

        // Forms
        "stc-form",
        "wtc-form",

        // Generic permissions
        "feed-marks",
        "trainee-profile"
    ],
    operator: [
        // Limited Candidate Management
        "add-candidate",
        "manage-candidates",

        // Forms - basic data entry
        "stc-form",
        "wtc-form",

        // View-only permissions
        "view-candidates",
        "view-marks",
        "view-attendance",
    ]
}

```

```

        "view-profiles"
    ]
};

module.exports = {
    PORT,
    CORS_ORIGIN,
    NODE_ENV,
    SESSION_EXPIRY,
    rolePermissions,
};

```

Key Features:

- Defines server settings; port, CORS origin, and environment mode, with environment variable support.
- Sets session expiry duration to 24 hours.
- Implements a rolePermissions object mapping user role to their allowed actions and features.
- Supports role-based access control for features like candidate management, STC/WTC/Non-Railway modules, forms, and generic UI elements.
- Exports all configuration values and permissions for use throughout the backend.

3.4 MIDDLEWARE LAYER

3.4.1 Authentication Middleware

Provides middleware for authenticating API requests using token-based sessions. It checks for an access token in the Authorization header of incoming requests, validates the token against an in-memory session store, and ensures the session has not expired. If the token is missing, invalid, or expired, the middleware responds with an appropriate error. For valid sessions, it extends the session expiry by 24 hours and attaches the user information to the request object, allowing downstream handlers to access authenticated user data.

File: middleware/auth.js

```

const { sessions } = require('../utils/session');

// Authentication middleware to check and validate tokens
const authenticateToken = (req, res, next) => {
    const authHeader = req.headers['authorization'];
    const token = authHeader && authHeader.split(' ')[1];

    if (!token) {

```

```

        return res.status(401).json({ message: 'Access token required'
    });
    }

    const session = sessions.get(token);
    if (!session || session.expires < Date.now()) {
        sessions.delete(token);
        return res.status(403).json({ message: 'Invalid or expired token'
    });
    }

    // Extend session for active users
    session.expires = Date.now() + (24 * 60 * 60 * 1000); // 24 hours
    req.user = session.user;
    next();
};

module.exports = { authenticateToken };

```

Key Features:

- **Session-Based Authentication:** Uses in-memory sessions instead of JWT tokens
- **Token Validation:** Extracts and validates Bearer tokens from Authorization header
- **Session Expiration:** Automatically removes expired sessions
- **Session Extension:** Extends active sessions by 24 hours on each request
- **User Context:** Attaches user information to request object for downstream use
- **Error Handling:** Returns appropriate HTTP status codes for authentication failures

Authentication Flow:

1. **Token Extraction:** Gets token from Authorization: Bearer <token> header
2. **Session Lookup:** Checks if token exists in active sessions
3. **Expiration Check:** Validates session hasn't expired
4. **Session Cleanup:** Removes expired sessions automatically
5. **Session Extension:** Extends valid sessions by 24 hours
6. **User Context:** Adds user data to request object
7. **Route Protection:** Calls next() for valid sessions, returns error for invalid ones
8. **Usage Pattern:** Adds user data to request object
9. **Route Protection:** Calls next() for valid sessions, returns error for invalid ones

Usage Pattern:

```

// Protected route example
router.get('/protected-route', authenticateToken, (req, res) => {
    // req.user contains authenticated user information
    res.json({ message: 'Access granted', user: req.user });
});

```

```
});
```

3.4.2 Image Upload Middleware

Configures and exports middleware for handling image uploads in the backend using the multer library. It sets up a disk storage engine that saves uploaded files to a temporary directory, ensuring the directory exists before storing files. Each uploaded file is assigned a unique temporary filename, with the expectation that controllers will later rename and move the files as needed. The middleware strictly allows only image files to be uploaded, rejecting other file types, and enforces a maximum file size of 1MB. This approach helps maintain security, organization, and control over file uploads in the application.

File: middleware/upload.js

```
const multer = require('multer');
const path = require('path');
const fs = require('fs');

// Configure multer for file uploads
const storage = multer.diskStorage({
  destination: function (req, file, cb) {
    const tempDir = path.join(__dirname, '../uploads/temp');
    // Ensure the temporary directory exists
    if (!fs.existsSync(tempDir)) {
      fs.mkdirSync(tempDir, { recursive: true });
    }
    // Temporary storage - will be moved by controller
    cb(null, tempDir);
  },
  filename: function (req, file, cb) {
    // Temporary filename - will be renamed by controller
    const uniqueSuffix = Date.now() + '-' + Math.round(Math.random()
* 1E9);
    cb(null, 'temp-' + uniqueSuffix +
path.extname(file.originalname));
  }
});

// File filter
const fileFilter = (req, file, cb) => {
  // Check file type
  if (file.mimetype.startsWith('image/')) {
    cb(null, true);
  } else {
    cb(new Error('Only image files are allowed!'), false);
  }
};

const upload = multer({
  storage: storage,
  limits: {
```

```

        fileSize: 1 * 1024 * 1024 // 1MB limit
      },
      fileFilter: fileFilter
    });

module.exports = upload;

```

Key Features:

- Uses multer for processing file uploads.
- Configures disk storage to a temporary uploads' directory, creating it if it doesn't exist.
- Assigns unique temporary filenames to each uploaded file.
- Restricts uploads to image files only through a custom file filter.
- Enforces a 1MB maximum file size limit.
- Exports the configured upload middleware for integration with backend routes and controllers.

3.5 UTILITIES LAYER

3.5.1 Session Management

Implements an in-memory session management system for the backend. It provides functions to create, retrieve, and remove user sessions, each identified by a unique token and associated with an expiry time. Sessions are stored in a JavaScript Map for fast access. The utility also includes a mechanism to periodically clean up expired sessions, ensuring efficient memory usage. This module is essential for handling authentication and maintaining user state across requests in the application.

File: utils/session.js

```

const { SESSION_EXPIRY } = require('../config/config');

// Session storage (in-memory for simplicity)
const sessions = new Map();

// Helper function to generate session token
const generateSessionToken = () => {
  return Math.random().toString(36).substring(2) +
    Date.now().toString(36);
};

// Create a new session
const createSession = (user) => {
  const token = generateSessionToken();
  const expires = Date.now() + SESSION_EXPIRY;

```

```

    // Store session
    sessions.set(token, { user, expires });

    return { token, expires };
};

// Remove a session
const removeSession = (token) => {
    if (sessions.has(token)) {
        sessions.delete(token);
        return true;
    }
    return false;
};

// Clean up expired sessions
const cleanSessions = () => {
    const now = Date.now();
    let cleanedCount = 0;

    sessions.forEach((session, token) => {
        if (session.expires < now) {
            sessions.delete(token);
            cleanedCount++;
        }
    });

    return cleanedCount;
};

// Schedule regular cleanup every hour
setInterval(cleanSessions, 60 * 60 * 1000);

module.exports = {
    sessions,
    createSession,
    removeSession,
    cleanSessions
};

```

Key Features:

- **In-Memory Storage:** Uses JavaScript Map for fast session lookup
- **Token Generation:** Creates unique tokens using random strings and timestamps
- **Session Expiration:** Automatic cleanup of expired sessions
- **Memory Management:** Hourly cleanup prevents memory leaks
- **Simple API:** Clean interface for session operations

3.5.2 Unique Ticket Number Generator

Provides a robust utility for generating unique ticket numbers for candidates based on their designation and trainee type. It uses a queue and mutex-like logic to prevent race conditions during ticket generation, ensuring that each ticket number is unique and sequential. The utility supports custom prefix logic, including special handling for certain designations, and uses acronyms for others. It queries the database to determine the highest existing ticket number for a given prefix, increments it, and generates a new ticket number with zero-padded formatting. The module is designed to be asynchronous and safe for concurrent use, making it suitable for high-traffic environments.

File: `utils/ticketGenerator.js`

```
const sqlite3 = require('sqlite3').verbose();
const { db } = require('../config/db');

// Mutex to prevent race conditions in ticket generation
let ticketGenerationQueue = [];
let isProcessing = false;
// Track highest generated numbers per prefix to avoid duplicates
let generatedNumbers = {};

// Helper function to create an acronym from a phrase
const createAcronym = (phrase) => {
  if (!phrase || typeof phrase !== 'string') {
    return 'TKT'; // Return a default prefix if phrase is invalid
  }
  // Words to ignore when creating the acronym
  const ignoreWords = ['for', 'of', 'in', 'a', 'an', 'the'];
  return phrase
    .split(' ')
    .filter(word => !ignoreWords.includes(word.toLowerCase()))
    .map(word => word.charAt(0))
    .join('')
    .toUpperCase();
};

const generateTicketNumber = async (designation, traineeType) => {
  return new Promise((resolve, reject) => {
    console.log(`Queuing ticket generation for designation:
    ${designation} in ${traineeType}`);

    if (!traineeType) {
      reject(new Error("Trainee type is required to generate a
      ticket number."));
      return;
    }

    // Add to queue
    ticketGenerationQueue.push({
      designation,
      traineeType,
      resolve,

```

```

        reject
    });

    // Process queue if not already processing
    if (!isProcessing) {
        processTicketQueue();
    }
    });
};

const processTicketQueue = async () => {
    if (isProcessing || ticketGenerationQueue.length === 0) {
        return;
    }

    isProcessing = true;

    while (ticketGenerationQueue.length > 0) {
        const { designation, traineeType, resolve, reject } =
            ticketGenerationQueue.shift();

        try {
            const ticketNumber = await
                generateTicketNumberInternal(designation, traineeType);
            resolve(ticketNumber);
        } catch (error) {
            reject(error);
        }
    }

    isProcessing = false;
};

const generateTicketNumberInternal = async (designation, traineeType) =>
{
    return new Promise((resolve, reject) => {
        console.log(`Generating ticket number for designation:
            ${designation} in ${traineeType}`);

        let prefix;
        // Special handling for WTC designations
        if (traineeType === 'wtc') {
            // Handle specific cases
            switch(designation) {
                case 'CG Apprentice':
                    prefix = 'CG';
                    break;
                case 'RRB Apprentice Technician III':
                case 'GDCE App. Tech. III':
                    prefix = 'TECH';
                    break;
                case 'RRC Act Apprentice 1961':
                    prefix = 'ATR';
                    break;
                case 'RRC Assistant Workshop':
                case 'CG Assistant Workshop':
                    prefix = 'HK';

```

```

        break;
      case 'Act Junior Apprentices':
        prefix = 'AHA';
        break;
      default:
        // For all other cases, use acronym
        prefix = createAcronym(designation);
    }
  } else {
    // For non-WTC candidates, keep existing logic
    prefix = traineeType === 'stc' ? designation.toUpperCase() :
createAcronym(designation);
  }

  const tableName = `${traineeType}_candidates`;
  const prefixKey = `${traineeType}_${prefix.toLowerCase()}`;

  const query = `
    SELECT ticket_no
    FROM ${tableName}
    WHERE LOWER(ticket_no) LIKE LOWER(?) || '%'
    ORDER BY ticket_no
  `;

  db.all(query, [`${prefix}`], (err, rows) => {
    if (err) {
      console.error(`Error fetching ticket numbers for
${designation}:`, err.message);
      reject(err);
      return;
    }

    let maxNumber = 0;
    const prefixLower = prefix.toLowerCase();

    if (rows && rows.length > 0) {
      rows.forEach(row => {
        const ticket = row.ticket_no;
        const ticketLower = ticket.toLowerCase();

        if (ticketLower.startsWith(prefixLower)) {
          const numericPart = ticket.slice(prefix.length);
          const number = parseInt(numericPart, 10);

          if (!isNaN(number) && number > maxNumber) {
            maxNumber = number;
          }
        }
      });
    }

    if (generatedNumbers[prefixKey] &&
generatedNumbers[prefixKey] > maxNumber) {
      maxNumber = generatedNumbers[prefixKey];
    }

    const newNumber = maxNumber + 1;

```

```

        // Use 5 digits padding for the numeric part
        const newTicketNumber =
`${prefix}${String(newNumber).padStart(5, '0')}`;

        generatedNumbers[prefixKey] = newNumber;

        console.log(`Generated new ticket number: ${newTicketNumber}
(prefix: ${prefix}, max found: ${maxNumber})`);
        resolve(newTicketNumber);
    });
});
};

module.exports = {
    generateTicketNumber,
};

```

Key Features:

- Generates unique, sequential ticket numbers for candidates based on designation and trainee type.
- Uses a queue and processing flag to prevent race conditions and ensure thread safety.
- Supports custom prefix logic, including special cases and acronym generation.
- Queries the database to find the current highest ticket number for a given prefix.
- Ensures ticket numbers are zero-padded to five digits.
- Tracks generated numbers in-memory to avoid duplicates within a session.
- Exports an asynchronous function for ticket number generation, suitable for use in controllers and services.

3.6 CONTROLLERS LAYER

3.6.1 Authentication Controller

Handles user authentication operations including login, logout, and session management.

File: controllers/authController.js

```

const User = require('../models/User.js');
const { createSession, removeSession } = require('../utils/session.js');

// Login controller
const login = async (req, res) => {
    try {
        const { role, password } = req.body;

        // Input validation
        if (!role || !password) {

```

```

        return res.status(400).json({ message: 'Role and password are
required' });
    }

    // Role validation
    if (!['admin', 'master', 'operator'].includes(role)) {
        return res.status(400).json({ message: 'Invalid role' });
    }

    // Verify user credentials
    const user = await User.verify(role, password);

    if (!user) {
        return res.status(401).json({ message: 'Invalid credentials'
});
    }

    // Create session
    const { token } = createSession(user);

    // Return success response
    res.json({
        success: true,
        token,
        user
    });
} catch (error) {
    console.error('Login error:', error);
    res.status(500).json({ message: 'Internal server error' });
}
};

// Verify token controller
const verifyToken = (req, res) => {
    // Authentication middleware already validated the token
    res.json({
        success: true,
        user: req.user
    });
};

// Logout controller
const logout = (req, res) => {
    try {
        const authHeader = req.headers['authorization'];
        const token = authHeader && authHeader.split(' ')[1];

        if (token) {
            removeSession(token);
        }

        res.json({ success: true, message: 'Logged out successfully' });
    } catch (error) {
        console.error('Logout error:', error);
        res.status(500).json({ message: 'Internal server error' });
    }
};

```

```
// Get user permissions
const getPermissions = (req, res) => {
  try {
    res.json({
      role: req.user.role,
      permissions: req.user.permissions
    });
  } catch (error) {
    console.error('Get permissions error:', error);
    res.status(500).json({ message: 'Internal server error' });
  }
};

module.exports = {
  login,
  verifyToken,
  logout,
  getPermissions
};
```

Key Features:

- **Role-Based Authentication:** Supports admin, master, and operator roles
- **Input Validation:** Validates required fields and role types
- **Session Management:** Creates and removes sessions using utility functions
- **Error Handling:** Comprehensive error handling with appropriate HTTP status codes
- **Security:** Validates credentials before creating sessions

3.6.2 Database Export to Excel

Implements functionality to export the entire SQLite database to an Excel file. It retrieves all table names and their data, as well as foreign key relationships, to organize the export process. Main tables (those referenced by foreign keys) are processed first, followed by the remaining tables. For each table, a worksheet is created in the Excel file, with headers and formatted rows. The resulting Excel file is then sent as a downloadable response to the client. The controller uses the ExcelJS library for Excel file creation and handles errors gracefully, providing informative responses in case of failure.

File: controllers/exportController.js

```
const ExcelJS = require('exceljs');
const { db } = require('../config/db');

// Function to get all table names in the database
const getAllTables = () => {
  return new Promise((resolve, reject) => {
```

```

        const sql = `SELECT name FROM sqlite_master WHERE type='table'
AND name NOT LIKE 'sqlite_%'`;
        db.all(sql, [], (err, rows) => {
            if (err) {
                reject(err);
            } else {
                resolve(rows.map(row => row.name));
            }
        });
    });
};

// Function to get all data from a specific table
const getTableData = (tableName) => {
    return new Promise((resolve, reject) => {
        const sql = `SELECT * FROM ${tableName}`;
        db.all(sql, [], (err, rows) => {
            if (err) {
                reject(err);
            } else {
                resolve(rows);
            }
        });
    });
};

// Function to get foreign key relationships
const getForeignKeyRelationships = () => {
    return new Promise((resolve, reject) => {
        // This query needs to be corrected to use the actual column
        // names from pragma_foreign_key_list
        const sql = `SELECT m.name as table_name, p."table" as
parent_table
FROM sqlite_master m
JOIN pragma_foreign_key_list(m.name) p
WHERE m.type='table'`;
        db.all(sql, [], (err, rows) => {
            if (err) {
                reject(err);
            } else {
                const relationships = {};
                rows.forEach(row => {
                    if (!relationships[row.parent_table]) {
                        relationships[row.parent_table] = [];
                    }
                    relationships[row.parent_table].push(row.table_name);
                });
                resolve(relationships);
            }
        });
    });
};

// Export database to Excel
exports.exportDatabase = async (req, res) => {
    try {
        // Create a new Excel workbook

```

```

const workbook = new ExcelJS.Workbook();
workbook.creator = 'STVT System';
workbook.lastModifiedBy = 'STVT Export Tool';
workbook.created = new Date();
workbook.modified = new Date();

// Get all tables
const tables = await getAllTables();

// Get foreign key relationships
const relationships = await getForeignKeyRelationships();

// Process main tables first (those that are referenced by
foreign keys)
const mainTables = Object.keys(relationships);
const processedTables = new Set();

// Process main tables first
for (const tableName of mainTables) {
    if (processedTables.has(tableName)) continue;

    // Get data from the table
    const data = await getTableData(tableName);

    // Create a new worksheet
    const worksheet = workbook.addWorksheet(tableName);

    // Add headers if data exists
    if (data.length > 0) {
        const headers = Object.keys(data[0]);
        worksheet.addRow(headers);

        // Add data rows
        data.forEach(row => {
            worksheet.addRow(Object.values(row));
        });

        // Format headers
        const headerRow = worksheet.getRow(1);
        headerRow.font = { bold: true, color: { argb: 'FFFFFFFF'
        fgColor: { argb: 'FF0070C0' } } };

        headerRow.fill = { type: 'pattern', pattern: 'solid',
        fgColor: { argb: 'FF0070C0' } } };

        // Auto fit columns
        worksheet.columns.forEach(column => {
            column.width = Math.max(15, Math.min(30, column.width
|| 15));
        });

        processedTables.add(tableName);
    }

    // Process remaining tables
    for (const tableName of tables) {
        if (processedTables.has(tableName)) continue;

```



```

// Get data from the table
const data = await getTableData(tableName);

// Create a new worksheet
const worksheet = workbook.addWorksheet(tableName);

// Add headers if data exists
if (data.length > 0) {
  const headers = Object.keys(data[0]);
  worksheet.addRow(headers);

  // Add data rows
  data.forEach(row => {
    worksheet.addRow(Object.values(row));
  });

  // Format headers
  const headerRow = worksheet.getRow(1);
  headerRow.font = { bold: true, color: { argb: 'FFFFFFFF'
  } };

  headerRow.fill = { type: 'pattern', pattern: 'solid',
  fgColor: { argb: 'FF0070C0' } };

  // Auto fit columns
  worksheet.columns.forEach(column => {
    column.width = Math.max(15, Math.min(30, column.width
    || 15));
  });
}

// Write to buffer
const buffer = await workbook.xlsx.writeBuffer();

// Set headers for file download
res.setHeader('Content-Type', 'application/vnd.openxmlformats-
officedocument.spreadsheetml.sheet');
res.setHeader('Content-Disposition', `attachment;
filename=STVT_Database_${new Date().toISOString().slice(0, 10)}.xlsx`);

// Send response
res.send(Buffer.from(buffer));
} catch (error) {
  console.error('Error exporting database:', error);
  res.status(500).json({ success: false, message: 'Failed to export
database', error: error.message });
}
};

```

Key Features:

- Retrieves all table names from the SQLite database.
- Fetches all data from each table for export.
- Identifies and processes main tables first based on foreign key relationships.

- Creates a worksheet for each table in the Excel file, including headers and formatted data rows.
- Uses the ExcelJS library to generate and format the Excel workbook.
- Sets appropriate HTTP headers to trigger file download in the client.
- Handles errors and sends informative error responses if the export fails.

3.6.3 Line Training Controller

Manages all backend operations related to line training programs. It enables the creation, retrieval, updating, and deletion of line training records, supporting both single and multiple candidates at once. The controller groups training programs by activity center and date range, and provides both manual and automatic status management based on program dates. It includes helper methods for parsing status and descriptions from remarks, and allows for status updates across entire training groups. Additionally, it supports fetching records by ticket number and implements robust error handling and logging throughout its operations, ensuring reliable and maintainable API endpoints for line training management.

File: controllers/lineTrainingController.js

```
const { generateTicketNumber } = require("../utils/ticketGenerator");

class LineTrainingController {
  constructor(lineTrainingModel) {
    this.lineTrainingModel = lineTrainingModel;
  }

  // Create records (one per ticket)
  async createCandidate(req, res) {
    try {
      const data = req.body;
      const tickets = Array.isArray(data.ticketNumbers)
        ? data.ticketNumbers
        : [data.ticketNumbers];

      const created = [];
      for (const no of tickets) {
        const cand = {
          ticket_no: no,
          activity_centre: data.activityCentre,
          start_date: data.startDate,
          end_date: data.endDate,
          remark: data.description || "",
        };
        created.push(await this.lineTrainingModel.create(cand));
      }

      return res.status(201).json({
        success: true,
      });
    } catch (error) {
      // Error handling logic
    }
  }
}
```

```

        message: `Line Training created for ${created.length}
candidate(s)`,
        data: {
            id: created[0]?.id,
            activityCentre: data.activityCentre,
            startDate: data.startDate,
            endDate: data.endDate,
            description: data.description,
            ticketNumbers: tickets,
        },
    });
} catch (err) {
    console.error("Error creating Line Training:", err);
    return res.status(500).json({ success: false, message: err.message
});
}
}

// Helper to parse status from remark
parseStatusFromRemark(remark) {
    if (!remark) return null;
    if (remark.startsWith("STATUS:")) {
        const parts = remark.split("|");
        return parts[0].replace("STATUS:", "").trim();
    }
    return null;
}

// Helper to get description without status
getDescriptionFromRemark(remark) {
    if (!remark) return "";
    if (remark.startsWith("STATUS:")) {
        const parts = remark.split("|");
        return parts.length > 1 ? parts.slice(1).join("|") : "";
    }
    return remark;
}

// List grouped by centre + dates
async getCandidates(req, res) {
    try {
        const all = await this.lineTrainingModel.getAll();
        const programs = {};

        all.forEach((c) => {
            const key = `${c.activity_centre}_${c.start_date}_${c.end_date}`;
            if (!programs[key]) {
                // Parse status from remark or use auto-calculated status
                const manualStatus = this.parseStatusFromRemark(c.remark);
                const autoStatus = this.determineStatus(c.start_date,
c.end_date);

                programs[key] = {
                    id: c.id,
                    activityCentre: c.activity_centre,
                    startDate: c.start_date,
                    endDate: c.end_date,

```

```

        description: this.getDescriptionFromRemark(c.remark),
        status: manualStatus || autoStatus, // Use manual status if
available, otherwise auto
        ticketNumbers: [],
    };
    }
    programs[key].ticketNumbers.push(c.ticket_no);
});

return res.json({
    success: true,
    message: "Line Training Programs retrieved successfully",
    data: Object.values(programs),
    count: Object.values(programs).length,
});
} catch (err) {
    console.error("Error getting Line Training programs:", err);
    return res
        .status(500)
        .json({ success: false, message: "Failed to retrieve" });
}
}

// Compute auto-status if needed
determineStatus(start, end) {
    const today = new Date().setHours(0, 0, 0, 0);
    const s = new Date(start).setHours(0, 0, 0, 0);
    const e = new Date(end).setHours(0, 0, 0, 0);
    if (today < s) return "Scheduled";
    if (today <= e) return "In Progress";
    return "Completed";
}

// Get one program by record-ID
async getCandidateById(req, res) {
    try {
        const { id } = req.params;
        const c = await this.lineTrainingModel.getById(id);
        if (!c) {
            return res
                .status(404)
                .json({ success: false, message: "Training not found" });
        }

        const all = await this.lineTrainingModel.getAll();
        const group = all.filter(
            (x) =>
                x.activity_centre === c.activity_centre &&
                x.start_date === c.start_date &&
                x.end_date === c.end_date
        );

        // Parse status from remark or use auto-calculated status
        const manualStatus = this.parseStatusFromRemark(c.remark);
        const autoStatus = this.determineStatus(c.start_date, c.end_date);

        return res.json({

```

```

        success: true,
        message: "Training program retrieved successfully",
        data: {
            id,
            activityCentre: c.activity_centre,
            startDate: c.start_date,
            endDate: c.end_date,
            description: this.getDescriptionFromRemark(c.remark),
            status: manualStatus || autoStatus, // Use manual status if
available, otherwise auto
            ticketNumbers: group.map((x) => x.ticket_no),
        },
    });
} catch (err) {
    console.error("Error getting training program:", err);
    return res
        .status(500)
        .json({ success: false, message: "Failed to retrieve" });
}
}

// Update entire program (delete + recreate)
async updateCandidate(req, res) {
    try {
        const { id } = req.params;
        const data = req.body;

        // find existing record
        const existing = await this.lineTrainingModel.getById(id);
        if (!existing) {
            return res
                .status(404)
                .json({ success: false, message: "Training not found" });
        }

        // remove old group
        const all = await this.lineTrainingModel.getAll();
        const group = all.filter(
            (x) =>
                x.activity_centre === existing.activity_centre &&
                x.start_date === existing.start_date &&
                x.end_date === existing.end_date
        );
        for (const x of group) {
            await this.lineTrainingModel.deleteByTicketNumber(x.ticket_no);
        }

        // recreate
        const tickets = Array.isArray(data.ticketNumbers)
            ? data.ticketNumbers
            : [data.ticketNumbers];
        const created = [];
        for (const no of tickets) {
            const cand = {
                ticket_no: no,
                activity_centre: data.activityCentre,
                start_date: data.startDate,
            };

```

```

        end_date: data.endDate,
        remark: data.description || "",
    };
    created.push(await this.lineTrainingModel.create(cand));
}

return res.json({
    success: true,
    message: "Training program updated successfully",
    data: {
        id: created[0]?.id,
        activityCentre: data.activityCentre,
        startDate: data.startDate,
        endDate: data.endDate,
        ticketNumbers: tickets,
    },
});
} catch (err) {
    console.error("Error updating training program:", err);
    return res.status(500).json({ success: false, message: err.message
});
}
}

// Delete entire program by ID
async deleteCandidate(req, res) {
    try {
        const { id } = req.params;
        const existing = await this.lineTrainingModel.getById(id);
        if (!existing) {
            return res
                .status(404)
                .json({ success: false, message: "Training not found" });
        }

        const all = await this.lineTrainingModel.getAll();
        const group = all.filter(
            (x) =>
                x.activity_centre === existing.activity_centre &&
                x.start_date === existing.start_date &&
                x.end_date === existing.end_date
        );
        for (const x of group) {
            await this.lineTrainingModel.deleteByTicketNumber(x.ticket_no);
        }

        return res.json({
            success: true,
            message: "Training program deleted successfully",
        });
    } catch (err) {
        console.error("Error deleting training program:", err);
        return res
            .status(500)
            .json({ success: false, message: "Failed to delete" });
    }
}
}

```

```

// — STATUS UPDATE —
async updateTrainingStatus(req, res) {
  try {
    const { id } = req.params;
    const { status } = req.body;

    console.log(`=== STATUS UPDATE START ===`);
    console.log(`ID: ${id}, Status: ${status}`);

    const valid = [
      "Scheduled",
      "In Progress",
      "Completed",
      "Cancelled",
      "On Hold",
    ];
    if (!valid.includes(status)) {
      return res.status(400).json({
        success: false,
        message: "Invalid status. Must be one of: " + valid.join(", "),
      });
    }

    // find one
    const c = await this.lineTrainingModel.getById(id);
    if (!c) {
      return res
        .status(404)
        .json({ success: false, message: "Training not found" });
    }

    console.log(`Found training:`, c);

    // update entire group - only update remark field
    const all = await this.lineTrainingModel.getAll();
    const group = all.filter(
      (x) =>
        x.activity_centre === c.activity_centre &&
        x.start_date === c.start_date &&
        x.end_date === c.end_date
    );

    console.log(`Found ${group.length} members in group`);

    for (const m of group) {
      console.log(`Processing member: ${m.ticket_no}`);

      // Parse existing remark to preserve description
      let existingDesc = "";
      if (m.remark) {
        if (m.remark.startsWith("STATUS:")) {
          const parts = m.remark.split("|");
          if (parts.length > 1) {
            existingDesc = parts.slice(1).join("|");
          }
        } else {

```

```

        existingDesc = m.remark;
    }
}

// Build new remark with status
const newRemark = existingDesc
    ? `STATUS:${status}|${existingDesc}`
    : `STATUS:${status}`;

console.log(`Updating ${m.ticket_no} with remark: ${newRemark}`);

// Only update the remark field
await this.lineTrainingModel.updateByTicketNumber(m.ticket_no, {
    remark: newRemark,
});

console.log(`Successfully updated ${m.ticket_no}`);
}

console.log(`=== STATUS UPDATE COMPLETE ===`);

return res.json({
    success: true,
    message: "Training status updated successfully",
    data: { id, status },
});
} catch (err) {
    console.error("=== STATUS UPDATE ERROR ===", err);
    return res
        .status(500)
        .json({ success: false, message: "Failed to update status" });
}
}

// Get all line training records for a ticket number (excluding
designation)
async getAllByTicketNumber(req, res) {
    try {
        const { ticketNo } = req.params;
        const records = await this.lineTrainingModel.getAllByTicketNumber(
            ticketNo
        );
        return res.json({
            success: true,
            data: records,
            count: records.length,
        });
    } catch (err) {
        console.error("Error fetching line training by ticket:", err);
        return res
            .status(500)
            .json({
                success: false,
                message: "Failed to fetch line training info",
            });
    }
}
}

```



```
}
```

```
module.exports = LineTrainingController;
```

Key Features:

- **Candidate Creation:** Create line training records for one or multiple candidates with ticket numbers, activity centre, dates, and remarks.
- **Status Parsing:** Parse and extract status and description from the remark field for each record.
- **Grouped Listing:** List and group line training programs by activity centre and date range, aggregating ticket numbers and status.
- **Automatic Status Calculation:** Automatically determine training status (Scheduled, In Progress, Completed) based on dates.
- **Single Program Retrieval:** Retrieve details of a specific training program by record ID, including all related ticket numbers.
- **Program Update:** Update an entire training program by deleting old records and recreating them with new details.
- **Program Deletion:** Delete all records associated with a specific training program group.
- **Status Update:** Update the status for all records in a group by modifying the remark field.
- **Ticket-Based Record Retrieval:** Fetch all line training records for a specific ticket number.
- **Error Handling:** Comprehensive error handling and logging for all controller operations.

3.6.4 Monthly Attendance Controller

Manages the monthly attendance workflow for trainees, especially within the WTC module. It provides API endpoints to mark or update attendance for individual candidates, retrieve detailed attendance records, generate lists of months relevant to a candidate's training period, and summarize attendance statistics for groups of candidates. The controller also supports bulk attendance marking, allowing efficient updates for multiple candidates at once. Input validation and robust error handling are implemented throughout, ensuring data integrity and reliable attendance management.

File: controllers/monthlyAttendanceController.js

```
const MonthlyAttendanceModel =
require('../models/MonthlyAttendanceModel');
const WtcModel = require('../models/wtcModel');

// Mark monthly attendance for a trainee
exports.markMonthlyAttendance = async (req, res) => {
  try {
    const { candidateId, month, totalClasses, classesAttended } =
req.body;

    if (!candidateId || !month) {
      return res.status(400).json({
        success: false,
        message: 'Candidate ID and month are required'
      });
    }

    // Validate month format (YYYY-MM)
    const monthRegex = /^\d{4}-\d{2}$/;
    if (!monthRegex.test(month)) {
      return res.status(400).json({
        success: false,
        message: 'Month must be in YYYY-MM format'
      });
    }

    // At least one of total classes or classes attended must be
provided
    if (totalClasses === undefined || classesAttended === undefined)
{
      return res.status(400).json({
        success: false,
        message: 'Total classes and classes attended must be
provided'
      });
    }

    const markResult = await MonthlyAttendanceModel.markAttendance(
      candidateId,
      month,
      totalClasses,
      classesAttended
    );

    // After marking attendance, fetch the updated attendance data
    const updatedAttendanceData = await
MonthlyAttendanceModel.getMonthlyAttendanceByCandidate(candidateId);

    res.status(200).json({
      success: true,
      message: markResult.updated ? 'Monthly attendance updated
successfully' : 'Monthly attendance marked successfully',
      data: updatedAttendanceData
    });
  } catch (error) {
```

```

        console.error('Error marking monthly attendance:', error);
        res.status(500).json({
            success: false,
            message: error.message || 'Failed to mark monthly attendance'
        });
    }
};

// Get monthly attendance for a specific candidate
exports.getMonthlyAttendance = async (req, res) => {
    try {
        const { candidateId } = req.params;

        if (!candidateId) {
            return res.status(400).json({
                success: false,
                message: 'Candidate ID is required'
            });
        }

        const attendanceData = await
MonthlyAttendanceModel.getMonthlyAttendanceByCandidate(candidateId);

        res.status(200).json({
            success: true,
            data: attendanceData
        });
    } catch (error) {
        console.error('Error getting monthly attendance:', error);
        res.status(500).json({
            success: false,
            message: error.message || 'Failed to get monthly attendance'
        });
    }
};

// Get all months between joining and sparing dates for a candidate
exports.getAttendanceMonths = async (req, res) => {
    try {
        const { candidateId } = req.params;

        if (!candidateId) {
            return res.status(400).json({
                success: false,
                message: 'Candidate ID is required'
            });
        }

        const monthsData = await
MonthlyAttendanceModel.generateMonthsBetweenDates(candidateId);

        res.status(200).json({
            success: true,
            data: monthsData
        });
    } catch (error) {
        console.error('Error getting attendance months:', error);
    }
};

```

```

        res.status(500).json({
            success: false,
            message: error.message || 'Failed to get attendance months'
        });
    }
};

// Get monthly attendance summary for all candidates
exports.getMonthlyAttendanceSummary = async (req, res) => {
    try {
        const { batch, moduleNo } = req.query;

        const attendanceStats = await
MonthlyAttendanceModel.getMonthlyAttendanceStats(batch, moduleNo);

        res.status(200).json({
            success: true,
            data: attendanceStats
        });
    } catch (error) {
        console.error('Error getting monthly attendance summary:',
error);
        res.status(500).json({
            success: false,
            message: error.message || 'Failed to get monthly attendance
summary'
        });
    }
};

// Bulk mark monthly attendance for multiple candidates
exports.bulkMarkMonthlyAttendance = async (req, res) => {
    try {
        const { records } = req.body;

        if (!records || !Array.isArray(records) || records.length === 0)
        {
            return res.status(400).json({
                success: false,
                message: 'Valid attendance records array is required'
            });
        }

        // Validate each record has candidateId and month
        for (const record of records) {
            if (!record.candidateId || !record.month) {
                return res.status(400).json({
                    success: false,
                    message: 'Each record must contain candidateId and
month'
                });
            }
        }

        // Validate month format
        const monthRegex = /^\\d{4}-\\d{2}$/;
        if (!monthRegex.test(record.month)) {
            return res.status(400).json({

```

```

                success: false,
                message: `Invalid month format in record for
candidate ${record.candidateId}. Must be YYYY-MM`
            });
        }
    }

    const bulkResult = await
MonthlyAttendanceModel.bulkMarkMonthlyAttendance(records);

    res.status(200).json({
        success: true,
        message: `Bulk monthly attendance processed for
${bulkResult.results ? bulkResult.results.length : 0} records`,
        results: bulkResult.results || []
    });
} catch (error) {
    console.error('Error bulk marking monthly attendance:', error);
    res.status(500).json({
        success: false,
        message: 'Failed to process bulk monthly attendance',
        error: error.message
    });
}
};

```

Key Features:

- Marks or updates monthly attendance for individual candidates, with validation for required fields and date format.
- Retrieves monthly attendance records for a specific candidate.
- Generates a list of all months between a candidate's joining and sparing dates.
- Provides summary statistics of monthly attendance for all candidates, with optional filtering by batch or module.
- Supports bulk marking of monthly attendance for multiple candidates in a single request, with per-record validation.
- Implements comprehensive error handling and returns clear, structured API responses for all endpoints.

3.6.5 Candidate Management Controllers

The Nonrailway Controller, STC Controller, and WTC Controller files in the backend manage CRUD operations and specialized queries for Non-Railway, STC, and WTC candidates, respectively. Each controller supports creating candidates (with unique ticket number generation and image upload handling), retrieving candidates (by ticket number, designation, unit, or course type), updating and deleting candidate records, and additional features like resignation (STC) or backward compatibility methods. Image

uploads are managed by renaming and moving files to organized directories. The controllers ensure robust error handling and provide consistent API responses, making them central to the candidate management workflow in the application.

Key Features:

- **Ticket Number Generation:** Each candidate is assigned a unique ticket number using a centralized utility, ensuring no duplicates across the system.

```
const ticketNumber = await generateTicketNumber(
  candidateData.designation,
  "stc"
);
candidateData.ticket_no = ticketNumber;

if (req.file) {
  const imagePath = await this.handleImageUpload(
    req.file,
    ticketNumber,
    "stc"
  );
  candidateData.picture = imagePath;
}
```

- **Image Upload Handling:** Uploaded candidate images are renamed using the ticket number and stored in organized directories.

```
async handleImageUpload(file, ticketNumber, traineeType) {
  const oldPath = file.path;
  const fileExtension = path.extname(file.originalname);
  const newFileName = `${ticketNumber}${fileExtension}`;

  const uploadDir = path.join(__dirname, '../uploads',
traineeType);
  const newFullPath = path.join(uploadDir, newFileName);
  const relativePath = `uploads/${traineeType}/${newFileName}`;

  if (!fs.existsSync(uploadDir)) {
    fs.mkdirSync(uploadDir, { recursive: true });
  }

  fs.renameSync(oldPath, newFullPath);

  return relativePath;
}
```

- **CRUD Operations:**
 - **Create:** Add new candidates with all relevant details and image.
 - **Read:** Retrieve candidates by ticket number, designation, unit, or course type.
 - **Update:** Modify candidate details and update in database.

- **Delete:** Remove candidate records and associated images.
- **Specialized Queries:** Retrieve candidates by designation, unit, or course type for reporting and filtering.
- **Session and Training Data (STC):** STC candidates have additional session date management and resignation handling.
- **Backward Compatibility:** Methods are provided to support legacy API routes by mapping IDs to ticket numbers.

```
async getCandidateById(req, res) {
  req.params.ticketNumber = req.params.id;
  return this.getCandidateByTicketNumber(req, res);
}
```

- **Robust Error Handling:** All operations include try-catch blocks and return informative error messages.

3.6.6 Course Score Controllers

The course controllers manage the full lifecycle of candidate score, records for various technical and practical training modules. Each controller provides endpoints for creating, retrieving, updating, and deleting individual or bulk score records, as well as advanced features like upsert (create or update), session-wise marks retrieval, and detailed analysis of scores for each of the 12 courses STC curriculum.

Controllers validate the existence of related candidates (often in the STC module) before allowing score creation, ensuring data integrity. They also support supplementary condition and score in supplementary exams to clear the supplementary based on the criteria. Backward compatibility methods are included to support legacy API routes. All controllers implement robust error handling and return clear, structured API responses.

Key Features:

- **CRUD Operations:** Create, read, update, and delete score records for each course module, using ticket numbers as unique identifiers.
- **Candidate Validation:** Before creating or updating scores, controllers validate the existence (and sometimes designation) of the related in the STC module.

```
async createScore(req, res) {
  try {
    const scoreData = req.body;

    // Validate if STC candidate exists
    if (scoreData.ticket_no || scoreData.ticketNumber) {
```

```

        const ticketNo = scoreData.ticket_no ||
scoreData.ticketNumber;
        const stcCandidate = await
this.stcModel.getByTicketNumber(ticketNo);

        if (!stcCandidate) {
            return res.status(404).json({
                success: false,
                message: 'STC Candidate not found. Please
create STC candidate first.'
            });
        }

        const newScore = await this.mjiDModel.create(scoreData);
        res.status(201).json({
            success: true,
            message: 'MJI-D Score created successfully',
            data: newScore
        });
    } catch (error) {
        // error handling...
    }
}

```

- **Bulk Upload:** Accept arrays of score records for efficient batch creation, with per-record error reporting.

```

async bulkUploadScores(req, res) {
    try {
        const { scores } = req.body;

        if (!Array.isArray(scores) || scores.length === 0) {
            return res.status(400).json({
                success: false,
                message: 'Invalid scores data. Expected an array of
scores.'
            });
        }

        const results = [];
        const errors = [];

        for (let i = 0; i < scores.length; i++) {
            try {
                const score = scores[i];
                const newScore = await
this.mjiCwModel.create(score);
                results.push(newScore);
            } catch (error) {
                errors.push({
                    index: i,
                    ticket_no: scores[i].ticket_no ||
scores[i].ticketNumber,
                    error: error.message
                });
            }
        }
    }
}

```



```

    }
  }

  res.status(200).json({
    success: true,
    message: `Bulk upload completed. ${results.length}
scores created, ${errors.length} errors.`,
    data: {
      created: results,
      errors: errors,
      summary: {
        total: scores.length,
        created: results.length,
        failed: errors.length
      }
    }
  });
} catch (error) {
  // error handling...
}
}

```

- **Upsert Functionality:** Create a new score if none exists for a ticket number, or update the existing one.

```

async upsertScore(req, res) {
  try {
    const scoreData = req.body;
    const ticketNumber = scoreData.ticket_no ||
scoreData.ticketNumber;

    if (!ticketNumber) {
      return res.status(400).json({
        success: false,
        message: 'Ticket number is required'
      });
    }

    // Check if score exists
    const existingScore = await
this.mjiCwModel.getByTicketNumber(ticketNumber);

    let result;
    let message;

    if (existingScore) {
      // Update existing score
      result = await
this.mjiCwModel.updateByTicketNumber(ticketNumber, scoreData);
      message = 'MJI - C&W Score updated successfully';
    } else {
      // Create new score
      result = await this.mjiCwModel.create(scoreData);
      message = 'MJI - C&W Score created successfully';
    }
  }
}

```

```

        res.status(200).json({
            success: true,
            message: message,
            data: result,
            operation: existingScore ? 'updated' : 'created'
        });
    } catch (error) {
        // error handling...
    }
}

```

- **Session-wise Marks Retrieval:** Retrieve marks for specific sessions (e.g., session 1, 2, 3, 4) for a candidate, including total and maximum possible marks.

```

async getSessionMarks(req, res) {
    try {
        const { ticketNumber, session } = req.params;
        const score = await
this.mjiDModel.getByTicketNumber(ticketNumber);

        if (!score) {
            return res.status(404).json({
                success: false,
                message: 'MJI-D Score not found'
            });
        }

        let sessionMarks = {};
        switch(session) {
            case '1':
                sessionMarks = {
                    slp1_marks: score.slp1_marks,
                    slp2_marks: score.slp2_marks,
                    slp3_marks: score.slp3_marks,
                    slp4_marks: score.slp4_marks,
                    slp5_marks: score.slp5_marks,
                    slp6_marks: score.slp6_marks,
                    slp7_marks: score.slp7_marks,
                    total: (score.slp1_marks || 0) +
(score.slp2_marks || 0) + (score.slp3_marks || 0) +
                    (score.slp4_marks || 0) +
(score.slp5_marks || 0) + (score.slp6_marks || 0) + (score.slp7_marks ||
0),
                    max_total: 700
                };
                break;
            // ... other sessions
        }

        res.status(200).json({
            success: true,
            message: `MJI-D Session ${session} marks retrieved
successfully`,
            data: {
                ticket_no: ticketNumber,
                session: session,

```

```

        marks: sessionMarks
    }
    });
} catch (error) {
    // error handling...
}
}

```

- **Score Analysis:** Compute and return statistics such as highest, lowest, and average scores, pass counts, pass percentages, and per-paper stats for sessions and the overall course.

```

calculateSessionAnalysis(scores, sessionPrefix, papers, maxTotal) {
    const analysis = {
        total_candidates: scores.length,
        session_stats: {
            max_possible: maxTotal,
            highest_score: 0,
            lowest_score: maxTotal,
            average_score: 0,
            pass_count: 0,
            pass_percentage: 0
        },
        paper_stats: {}
    };

    let totalSessionScores = 0;
    const passThreshold = maxTotal * 0.5; // 50% pass mark

    scores.forEach(score => {
        let sessionTotal = 0;

        papers.forEach(paper => {
            const paperField = `${sessionPrefix}${paper}_marks`;
            const marks = score[paperField] || 0;
            sessionTotal += marks;

            if (!analysis.paper_stats[paper]) {
                analysis.paper_stats[paper] = {
                    highest: 0,
                    lowest: 100,
                    average: 0,
                    total: 0
                };
            }

            analysis.paper_stats[paper].highest =
Math.max(analysis.paper_stats[paper].highest, marks);
            analysis.paper_stats[paper].lowest =
Math.min(analysis.paper_stats[paper].lowest, marks);
            analysis.paper_stats[paper].total += marks;
        });

        totalSessionScores += sessionTotal;
        analysis.session_stats.highest_score =
Math.max(analysis.session_stats.highest_score, sessionTotal);
    });
}

```

```

        analysis.session_stats.lowest_score =
Math.min(analysis.session_stats.lowest_score, sessionTotal);

        if (sessionTotal >= passThreshold) {
            analysis.session_stats.pass_count++;
        }
    });

    // Calculate averages
    analysis.session_stats.average_score = totalSessionScores /
scores.length;
    analysis.session_stats.pass_percentage =
(analysis.session_stats.pass_count / scores.length) * 100;

    papers.forEach(paper => {
        analysis.paper_stats[paper].average =
analysis.paper_stats[paper].total / scores.length;
        delete analysis.paper_stats[paper].total;
    });

    return analysis;
}

```

- **Candidate-Score Details:** Fetch scores along with candidate details for richer reporting.
- **Backward Compatibility:** Methods to support legacy routes using IDs mapped to ticket numbers.
- **Robust Error Handling:** All endpoints include try-catch blocks and return informative error messages for API consumers.
- Different Controller for each course.

3.7 ROUTES LAYER

3.7.1 Authentication Routes

Defines API endpoints for authentication operations with proper middleware integration.

File: routes/authRoutes.js

```

const express = require('express');
const { login, verifyToken, logout, getPermissions } =
require('../controllers/authController');
const { authenticateToken } = require('../middleware/auth.js');

const router = express.Router();

// Auth routes

```

```

router.post('/login', login); // POST
/api/auth/login
router.get('/verify', authenticateToken, verifyToken); // GET
/api/auth/verify
router.post('/logout', authenticateToken, logout); // POST
/api/auth/logout
router.get('/permissions', authenticateToken, getPermissions); // GET
/api/auth/permissions

module.exports = router;

```

Route Breakdown:

- **POST /api/auth/login:** Public endpoint for user authentication
- **GET /api/auth/verify:** Protected endpoint to verify token validity
- **POST /api/auth/logout:** Protected endpoint for session termination
- **GET /api/auth/permissions:** Protected endpoint to get user role and permissions

Security Implementation:

- **Public Routes:** /login - No authentication required
- **Protected Routes:** /verify, /logout, /permissions - Require valid session token
- **Middleware Integration:** Uses authenticateToken middleware for route protection

3.7.2 Export Routes

Defines the routing for exporting the database in the backend application. It sets up an Express router that maps HTTP GET requests to the root path (/) to the exportDatabase function in the exportController. This enables clients to trigger a full database export via a single API endpoint.

File: routes/exportRoutes.js

```

const express = require('express');
const router = express.Router();
const exportController = require('../controllers/exportController');

// Export database route
router.get('/', exportController.exportDatabase);

module.exports = router;

```

Key Features:

- Sets up an Express router for export-related endpoints.
- Defines a GET route at / that triggers the database export process.

- Connects the route to the exportDatabase controller function.
- Exports the router for integration with the main application.

3.7.2 Line Training Routes

Defines the Express routes for managing line training programs in the backend. It connects various HTTP endpoints to the corresponding methods in the LineTrainingController, enabling CRUD operations, status updates, filtering, and legacy support for line training records. The routes support creating, retrieving, updating, and deleting line training programs, as well as filtering by designation, activity centre, and date range, and fetching records by ticket number.

File: routes/lineTrainingRoutes.js

```
const express = require("express");
const router = express.Router();
const LineTrainingController =
  require("../controllers/lineTrainingController");
const LineTrainingModel = require("../models/lineTraining");

const model = new LineTrainingModel();
const controller = new LineTrainingController(model);

// Main Routes
router.post("/", (req, res) => controller.createCandidate(req, res));
router.get("/", (req, res) => controller.getCandidates(req, res));

// Status update (must come before `/:id`)
router.put("/:id/status", (req, res) =>
  controller.updateTrainingStatus(req, res)
);

// Single-program CRUD
router.get("/:id", (req, res) => controller.getCandidateById(req, res));
router.put("/:id", (req, res) => controller.updateCandidate(req, res));
router.delete("/:id", (req, res) => controller.deleteCandidate(req,
res));

// Filters
router.get("/filter/designation/:designation", (req, res) =>
  controller.getCandidatesByDesignation(req, res)
);
router.get("/filter/activityCentre/:activityCentre", (req, res) =>
  controller.getCandidatesByActivityCentre(req, res)
);
router.get("/filter/dateRange", (req, res) =>
  controller.getCandidatesByDateRange(req, res)
);
```

```
// Fetch all line training records for a ticket number (excluding
designation)
router.get("/by-ticket/:ticketNo", (req, res) =>
  controller.getAllByTicketNumber(req, res)
);

// Legacy aliases (optional)
router.get("/legacy/:id", (req, res) => controller.getCandidateById(req,
res));
router.put("/legacy/:id", (req, res) => controller.updateCandidate(req,
res));
router.delete("/legacy/:id", (req, res) =>
  controller.deleteCandidate(req, res)
);

module.exports = router;
```

Key Features:

- Sets up RESTful routes for creating, listing, updating, and deleting line training programs.
- Provides a dedicated route for updating the status of a training program.
- Supports filtering of line training records by designation, activity centre, and date range.
- Allows retrieval of all line training records associated with a specific ticket number.
- Includes legacy routes for backward compatibility with older API patterns.
- Integrates the LineTrainingController and LineTrainingModel for handling business logic and data access.

3.7.3 Monthly Attendance Routes

Defines the Express routes for managing monthly attendance records of trainees. It maps various HTTP endpoints to the corresponding controller functions, enabling marking, retrieving, and summarizing attendance, as well as supporting bulk operations. The routes facilitate both individual and batch attendance management, and provide endpoints for retrieving attendance data and relevant months for each candidate.

File: routes/monthlyAttendanceRoutes.js

```
const express = require('express');
const router = express.Router();
const monthlyAttendanceController =
require('../controllers/monthlyAttendanceController');

// Mark monthly attendance for a trainee
router.post('/mark', monthlyAttendanceController.markMonthlyAttendance);
```

```
// Get monthly attendance for a specific candidate
router.get('/candidate/:candidateId',
monthlyAttendanceController.getMonthlyAttendance);

// Get all months between joining and sparing dates for a candidate
router.get('/months/:candidateId',
monthlyAttendanceController.getAttendanceMonths);

// Get monthly attendance summary for all candidates
router.get('/summary',
monthlyAttendanceController.getMonthlyAttendanceSummary);

// Bulk mark monthly attendance for multiple trainees
router.post('/bulk-mark',
monthlyAttendanceController.bulkMarkMonthlyAttendance);

module.exports = router;
```

Key Features:

- Route to mark or update monthly attendance for a single trainee.
- Route to retrieve monthly attendance records for a specific candidate.
- Route to get all months between a candidate's joining and sparing dates.
- Route to fetch summary statistics of monthly attendance for all candidates.
- Route to bulk mark monthly attendance for multiple trainees in one request.
- Integrates with the monthlyAttendanceController for business logic and data handling.

3.7.4 Candidate Management Routes

The nonRailwayRoutes.js, stcRoutes.js, and wtcRoutes.js files define the Express routing for candidate management in the Non-Railway, STC, and WTC modules, respectively. Each file sets up RESTful API endpoints for creating, retrieving, updating, and deleting candidate records, with support for image uploads, filtering, and backward compatibility. The routes are tightly integrated with their respective controllers and models, ensuring a clean separation of concerns and maintainable code structure. Additional endpoints support filtering candidates by designation, unit, or course type, and special routes exist for session management (STC) and legacy API compatibility.

Key Features:

- RESTful routes for CRUD operations on candidate records (create, read, update, delete).
- Image upload support for candidate profile pictures using middleware.


```
// Non-Railway, STC, WTC (pattern is the same)
router.post('/', upload.single('image'), (req, res) => {
  controller.createCandidate(req, res);
});
```

- Filtering endpoints for designation, unit, and course type.

```
router.get('/filter/designation/:designation', (req, res) => {
  controller.getCandidatesByDesignation(req, res);
});
router.get('/filter/unit/:unit', (req, res) => {
  controller.getCandidatesByUnit(req, res);
});
router.get('/filter/courseType/:courseType', (req, res) => {
  controller.getCandidatesByCourseType?.(req, res);
});
```

- Backward compatibility routes for legacy API consumers.

```
router.get('/legacy/:id', (req, res) => {
  controller.getCandidateById(req, res);
});
router.put('/legacy/:id', upload.single('image'), (req, res) => {
  controller.updateCandidate(req, res);
});
router.delete('/legacy/:id', (req, res) => {
  controller.deleteCandidate(req, res);
});
```

- Special endpoints for session management in STC (get/update session dates, resign candidate).

```
// Get session dates
router.get('/:ticketNumber/sessions', (req, res) => {
  stcController.getSessionDates(req, res);
});
// Update session dates
router.put('/:ticketNumber/sessions', (req, res) => {
  stcController.updateSessionDates(req, res);
});
// Resign candidate
router.put('/:ticketNumber/resign', (req, res) => {
  stcController.resignCandidate(req, res);
});
```

- Modular integration with controllers and models for business logic and data access.

3.7.5 Course Score Management Routes

The route files for the course modules define comprehensive Express routing for managing candidate scores in various technical and practical training programs. These routes support CRUD operations, paper-specific updates, session-wise and course-wide analytics, bulk uploads, upsert operations, and backward compatibility. Each route file is tightly integrated with its respective controller and model, ensuring modularity and maintainability. Also different route for different course ensures there is no overlap in course details.

Key Features:

- **CRUD Operations:** Define endpoints for creating, retrieving, updating, and deleting score records using ticket numbers.

```
// Create a new score
router.post('/', (req, res) => {
  controller.createScore(req, res);
});

// Get all scores
router.get('/', (req, res) => {
  controller.getScores(req, res);
});

// Get, update, or delete score by ticket number
router.get('/:ticketNumber', (req, res) => {
  controller.getScoreByTicketNumber(req, res);
});
router.put('/:ticketNumber', (req, res) => {
  controller.updateScoreByTicketNumber(req, res);
});
router.delete('/:ticketNumber', (req, res) => {
  controller.deleteScoreByTicketNumber(req, res);
});
```

- **Paper-Specific Updates:** Allow updating marks for individual papers within a candidate's score record.

```
router.put('/:ticketNumber/paper/:paperCode', (req, res) => {
  controller.updatePaperMarks(req, res);
});
```

- **Session-Wise and Course Analytics:** Provide endpoints for retrieving session-wise marks, session analysis, and overall course analysis for candidates.

```
// Get session-wise marks
router.get('/:ticketNumber/session/:session', (req, res) => {
  controller.getSessionMarks(req, res);
});

// Get session analysis
```

```

router.get('/analysis/session/:session', (req, res) => {
    controller.getSessionAnalysis(req, res);
});

// Get course overview analysis
router.get('/analysis/course/overview', (req, res) => {
    controller.getCourseAnalysis(req, res);
});

```

- **Bulk Operations:** Support bulk uploading of scores and upsert (create or update) operations for efficient data management.

```

// Bulk upload scores
router.post('/bulk/upload', (req, res) => {
    controller.bulkUploadScores(req, res);
});

// Upsert (create or update) score
router.post('/upsert', (req, res) => {
    controller.upsertScore(req, res);
});

```

- **Score Details with Candidate Info:** Offer routes to fetch scores along with associated candidate details for richer reporting.

```

// Get all scores with candidate details
router.get('/details/with-candidates', (req, res) => {
    controller.getScoresWithCandidateDetails(req, res);
});

// Get score with candidate details by ticket number
router.get('/:ticketNumber/details', (req, res) => {
    controller.getScoreWithCandidateDetails(req, res);
});

```

- **Backward Compatibility:** Include legacy routes for supporting older API patterns using candidate IDs.

```

router.get('/legacy/:id', (req, res) => {
    controller.getScoreById(req, res);
});
router.put('/legacy/:id', (req, res) => {
    controller.updateScore(req, res);
});
router.delete('/legacy/:id', (req, res) => {
    controller.deleteScore(req, res);
});

```

- Different route for each of the 12 courses.

3.8 MODEL LAYER

3.8.1 User Model

This defines the User model for the backend, focusing on user authentication and role-based permission assignment. It provides a static method to verify user credentials against the database and, upon successful authentication, returns a user object that includes the user's role and associated permissions.

File: models/User.js

```
const {db} = require ('../config/db.js');
const {rolePermissions} = require ('../config/config.js');

class User {
  // Verify user credentials
  static async verify(role, password) {
    return new Promise((resolve, reject) => {
      db.get(
        "SELECT * FROM users WHERE role = ? AND password = ?",
        [role, password],
        (err, row) => {
          if (err) {
            reject(err);
            return;
          }

          if (!row) {
            resolve(null);
            return;
          }

          // Format user object with permissions
          const user = {
            id: row.id,
            role: row.role,
            permissions: rolePermissions[row.role] || []
          };

          resolve(user);
        }
      );
    });
  }
}

module.exports= User;
```

Key Features:

- Verifies user credentials (role and password) against the database.
- Returns a user object containing the user's ID, role, and permissions if authentication is successful.
- Integrates role-based permissions from the central configuration.
- Handles errors and returns null if authentication fails.
- Exports the User class for use in authentication workflows.

3.8.2 Line Training Model

Defines the data model for managing line training candidate records in the backend. It handles database schema migration, table creation, and provides a comprehensive set of methods for CRUD operations, filtering, and data retrieval. The model ensures data integrity through unique constraints and foreign key relationships, and supports both modern and legacy API patterns.

File: models/lineTraining.js

```
const sqlite3 = require("sqlite3").verbose();
const { db } = require("../config/db");

class lineTrainingModel {
  constructor() {
    this.tableName = "line_training_candidates";
    this.migrateDatabaseSchema();
  }

  migrateDatabaseSchema() {
    // Check if table exists with old schema and migrate if needed
    db.get(`PRAGMA table_info(${this.tableName})`, (err, row) => {
      if (err) {
        // Table doesn't exist, create new one
        this.createTable();
      } else {
        // Check if we need to migrate by looking for the old UNIQUE
        constraint
        db.get(
          `SELECT sql FROM sqlite_master WHERE type='table' AND
          name='${this.tableName}'`,
          (err, result) => {
            if (
              result &&
              result.sql.includes("ticket_no TEXT UNIQUE NOT NULL")
            ) {
              console.log("Migrating line training table schema...");
              this.migrateFromOldSchema();
            } else {

```

```

        // Table already has new schema or doesn't exist
        this.createTable();
    }
}
);
}
});
}

migrateFromOldSchema() {
    // Backup existing data
    db.all(`SELECT * FROM ${this.tableName}`, (err, existingData) => {
        if (err) {
            console.error("Error reading existing data:", err);
            this.createTable();
            return;
        }

        // Drop old table
        db.run(`DROP TABLE IF EXISTS ${this.tableName}`, (err) => {
            if (err) {
                console.error("Error dropping old table:", err);
                return;
            }

            // Create new table with callback to restore data
            db.run(
                `CREATE TABLE IF NOT EXISTS ${this.tableName} (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    ticket_no TEXT NOT NULL,
                    name TEXT,
                    designation TEXT,
                    activity_centre TEXT,
                    start_date TEXT,
                    end_date TEXT,
                    remark TEXT,
                    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
                    updated_at DATETIME DEFAULT CURRENT_TIMESTAMP,
                    FOREIGN KEY(ticket_no) REFERENCES
stc_candidates(ticket_no),
                    UNIQUE(ticket_no, activity_centre, start_date,
end_date)
                )`,
                (err) => {
                    if (err) {
                        console.error("Error creating new table:", err);
                        return;
                    }

                    // Restore data (if any) after table is created
                    if (existingData && existingData.length > 0) {
                        console.log(`Restoring ${existingData.length} records...`);
                        existingData.forEach((record) => {
                            const sql = `INSERT INTO ${this.tableName} (
                                ticket_no, name, designation,
                                activity_centre, start_date, end_date, remark
                            ) VALUES (?, ?, ?, ?, ?, ?, ?)`;

```

```

        db.run(
            sql,
            [
                record.ticket_no,
                record.name,
                record.designation,
                record.activity_centre,
                record.start_date,
                record.end_date,
                record.remark,
            ],
            function (err) {
                if (
                    err &&
                    !err.message.includes("UNIQUE constraint failed")
                ) {
                    console.error("Error restoring record:", err);
                }
            }
        );
    });
    console.log("Migration completed successfully!");
}

}

);
});
});
}

createTable() {
    db.run(`CREATE TABLE IF NOT EXISTS ${this.tableName} (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        ticket_no TEXT NOT NULL,
        name TEXT,
        designation TEXT,
        activity_centre TEXT,
        start_date TEXT,
        end_date TEXT,
        remark TEXT,
        created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
        updated_at DATETIME DEFAULT CURRENT_TIMESTAMP,
        FOREIGN KEY(ticket_no) REFERENCES stc_candidates(ticket_no),
        UNIQUE(ticket_no, activity_centre, start_date, end_date)
    )`);
}

create(candidateData) {
    return new Promise((resolve, reject) => {
        const sql = `INSERT INTO ${this.tableName} (
            ticket_no, name, designation, activity_centre,
            start_date, end_date, remark
        ) VALUES (?, ?, ?, ?, ?, ?, ?)`;

        db.run(
            sql,
            [

```

```

        candidateData.ticket_no || candidateData.ticketNumber,
        candidateData.name,
        candidateData.designation,
        candidateData.activity_centre || candidateData.activityCentre,
        candidateData.start_date || candidateData.startDate,
        candidateData.end_date || candidateData.endDate,
        candidateData.remark,
    ],
    function (err) {
        if (err) {
            if (err.message.includes("UNIQUE constraint failed")) {
                reject(
                    new Error(
                        "Trainee is already enrolled in this training program"
                    )
                );
            } else {
                reject(err);
            }
        } else {
            resolve({
                id: this.lastID,
                ticket_no: candidateData.ticket_no ||
candidateData.ticketNumber,
                ...candidateData,
            });
        }
    }
);
});
}

getByTicketNumber(ticketNumber) {
    return new Promise((resolve, reject) => {
        const sql = `SELECT * FROM ${this.tableName} WHERE ticket_no = ?`;
        db.get(sql, [ticketNumber], (err, row) => {
            if (err) {
                reject(err);
            } else {
                resolve(row);
            }
        });
    });
}

updateByTicketNumber(ticketNumber, candidateData) {
    return new Promise((resolve, reject) => {
        // Build dynamic SQL based on what fields are provided
        const updates = [];
        const values = [];

        if (candidateData.name !== undefined) {
            updates.push("name = ?");
            values.push(candidateData.name);
        }
        if (candidateData.designation !== undefined) {
            updates.push("designation = ?");

```



```

        values.push(candidateData.designation);
    }
    if (
        candidateData.activity_centre !== undefined ||
        candidateData.activityCentre !== undefined
    ) {
        updates.push("activity_centre = ?");
        values.push(
            candidateData.activity_centre || candidateData.activityCentre
        );
    }
    if (
        candidateData.start_date !== undefined ||
        candidateData.startDate !== undefined
    ) {
        updates.push("start_date = ?");
        values.push(candidateData.start_date || candidateData.startDate);
    }
    if (
        candidateData.end_date !== undefined ||
        candidateData.endDate !== undefined
    ) {
        updates.push("end_date = ?");
        values.push(candidateData.end_date || candidateData.endDate);
    }
    if (candidateData.remark !== undefined) {
        updates.push("remark = ?");
        values.push(candidateData.remark);
    }
}

// Always update the timestamp
updates.push("updated_at = CURRENT_TIMESTAMP");

if (updates.length === 1) {
    // Only timestamp update
    resolve({ ticket_no: ticketNumber, ...candidateData });
    return;
}

const sql = `UPDATE ${this.tableName} SET ${updates.join(
    ", "
)} WHERE ticket_no = ?`;
values.push(ticketNumber);

db.run(sql, values, function (err) {
    if (err) {
        console.error("Update error:", err);
        reject(err);
    } else {
        if (this.changes > 0) {
            resolve({ ticket_no: ticketNumber, ...candidateData });
        } else {
            resolve(null);
        }
    }
});
});
});

```

```

    }

    deleteByTicketNumber(ticketNumber) {
      return new Promise((resolve, reject) => {
        const sql = `DELETE FROM ${this.tableName} WHERE ticket_no = ?`;
        db.run(sql, [ticketNumber], function (err) {
          if (err) {
            reject(err);
          } else {
            resolve(this.changes > 0);
          }
        });
      });
    }

    getAll() {
      return new Promise((resolve, reject) => {
        const sql = `SELECT * FROM ${this.tableName} ORDER BY created_at
DESC`;
        db.all(sql, [], (err, rows) => {
          if (err) {
            reject(err);
          } else {
            resolve(rows);
          }
        });
      });
    }

    getByDesignation(designation) {
      return new Promise((resolve, reject) => {
        const sql = `SELECT * FROM ${this.tableName} WHERE designation =
?`;
        db.all(sql, [designation], (err, rows) => {
          if (err) {
            reject(err);
          } else {
            resolve(rows);
          }
        });
      });
    }

    getByActivityCentre(activityCentre) {
      return new Promise((resolve, reject) => {
        const sql = `SELECT * FROM ${this.tableName} WHERE activity_centre
= ?`;
        db.all(sql, [activityCentre], (err, rows) => {
          if (err) {
            reject(err);
          } else {
            resolve(rows);
          }
        });
      });
    }
  }
}

```

```

    getByDateRange(startDate, endDate) {
      return new Promise((resolve, reject) => {
        const sql = `SELECT * FROM ${this.tableName} WHERE start_date >= ?
AND end_date <= ?`;
        db.all(sql, [startDate, endDate], (err, rows) => {
          if (err) {
            reject(err);
          } else {
            resolve(rows);
          }
        });
      });
    }

    // Get by record ID (primary key)
    getById(id) {
      return new Promise((resolve, reject) => {
        const sql = `SELECT * FROM ${this.tableName} WHERE id = ?`;
        db.get(sql, [id], (err, row) => {
          if (err) {
            reject(err);
          } else {
            resolve(row);
          }
        });
      });
    }

    // Get all line training records for a ticket number (excluding
    designation)
    getAllByTicketNumber(ticketNumber) {
      return new Promise((resolve, reject) => {
        const sql = `SELECT id, ticket_no, name, activity_centre,
start_date, end_date, remark, created_at, updated_at
FROM ${this.tableName} WHERE ticket_no = ? ORDER BY
created_at DESC`;
        db.all(sql, [ticketNumber], (err, rows) => {
          if (err) {
            reject(err);
          } else {
            resolve(rows);
          }
        });
      });
    }

    // Backward compatibility methods
    update(id, data) {
      return this.updateByTicketNumber(id, data);
    }
    delete(id) {
      return this.deleteByTicketNumber(id);
    }
  }

  module.exports = lineTrainingModel;

```

Key Features:

- Automatically migrates and creates the `line_training_candidates` table with the correct schema.
- Provides methods to create, retrieve, update, and delete candidate records by ticket number or ID.
- Supports filtering candidates by designation, activity centre, and date range.
- Ensures data integrity with unique constraints and foreign key references to the STC candidate's table.
- Offers methods to retrieve all records, records by ticket number, and records by group attributes.
- Includes backward compatibility methods for legacy API support.
- Handles schema migration from older table structures, preserving and restoring existing data.

3.8.3 Monthly Attendance Model

This file defines the data model for managing monthly attendance records of trainees, primarily for the WTC module. It creates and manages the `monthly_attendances` table, which tracks attendance per candidate per month, including total classes and classes attended. The model provides methods for marking and updating attendance, retrieving attendance records and statistics for individual candidates, generating lists of months between joining and sparring dates, bulk marking attendance for multiple candidates, and summarizing attendance statistics across batches or modules. Data integrity is ensured through unique constraints and foreign key relationships.

File: `models/MonthlyAttendanceModel.js`

```
const sqlite3 = require('sqlite3').verbose();
const { db } = require('../config/db');

class MonthlyAttendanceModel {
  constructor() {
    // Define table name explicitly
    this.tableName = 'monthly_attendances';
    this.createTable();
  }

  createTable() {
    db.run(`
      CREATE TABLE IF NOT EXISTS monthly_attendances (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        candidate_id INTEGER NOT NULL,
        ticket_no TEXT NOT NULL,
```

```

        month TEXT NOT NULL, -- Format: YYYY-MM
        total_classes INTEGER DEFAULT 0,
        classes_attended INTEGER DEFAULT 0,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        FOREIGN KEY (candidate_id) REFERENCES wtc_candidates(id)
ON DELETE CASCADE,
        UNIQUE(candidate_id, month)
    )
    `);
}

// Mark attendance for a trainee for a specific month
async markAttendance(candidateId, month, totalClasses = 0,
classesAttended = 0) {
    const tableName = this.tableName; // Store table name in local
    variable

    return new Promise((resolve, reject) => {
        // First check if the candidate exists in wtc_candidates
        db.get(
            'SELECT id, ticket_no FROM wtc_candidates WHERE id = ?',
            [candidateId],
            (err, candidate) => {
                if (err) return reject(err);
                if (!candidate) return reject(new Error('Candidate
not found'));

                // Check if a monthly attendance record already
                exists for this candidate
                db.get(
                    `SELECT id FROM ${tableName} WHERE candidate_id =
? AND month = ?`,
                    [candidateId, month],
                    (err, existing) => {
                        if (err) return reject(err);

                        if (existing) {
                            // Update existing record
                            const updateFields = [];
                            const updateValues = [];

                            // Update total classes and classes
                            attended if provided
                            if (totalClasses !== undefined &&
totalClasses >= 0) {
                                updateFields.push('total_classes =
?');
                                updateValues.push(totalClasses);
                            }

                            if (classesAttended !== undefined &&
classesAttended >= 0) {
                                updateFields.push('classes_attended =
?');
                                updateValues.push(classesAttended);
                            }
                        }
                    }
                )
            }
        )
    })
}

```

```

CURRENT_TIMESTAMP');

updateFields.push('updated_at =
updateValues.push(existing.id);

db.run(
  `UPDATE ${tableName} SET
  ${updateFields.join(', ')} WHERE id = ?`,
  [...updateValues],
  function (err) {
    if (err) return reject(err);

    // Fetch the updated record to
    return it
    db.get(
      `SELECT id, candidate_id,
      ticket_no, month, total_classes, classes_attended
      FROM ${tableName} WHERE id =
      ?`,
      [existing.id],
      (err, record) => {
        if (err) return
        reject(err);
        resolve({
          id: existing.id,
          updated: true,
          record: {
            id: record.id,
            candidateId:
            ticketNo:
            month:
            totalClasses:
            classesAttended:
            record.candidate_id,
            record.ticket_no,
            record.month,
            record.total_classes,
            record.classes_attended
          }
        });
      });
    });
  } else {
    // Insert new record
    db.run(
      `INSERT INTO ${tableName}
      (candidate_id, ticket_no, month,
      total_classes, classes_attended)
      VALUES (?, ?, ?, ?, ?)`,
      [candidateId, candidate.ticket_no,
      month, totalClasses, classesAttended],
      function (err) {
        if (err) return reject(err);

```



```

        db.all(
            `SELECT id, month, total_classes,
classes_attended
            FROM ${tableName}
            WHERE candidate_id = ?
            ORDER BY month ASC`,
            [candidateId],
            (err, records) => {
                if (err) return reject(err);

                // Calculate statistics
                const totalRecords = records.length;

                // Calculate total attendance across all
months
                let totalClassesAll = 0;
                let classesAttendedAll = 0;

                // Format the records
                const formattedRecords = records.map(record
=> {
                    totalClassesAll += record.total_classes
                    classesAttendedAll +=
record.classes_attended || 0;

                    return {
                        id: record.id,
                        month: record.month,
                        totalClasses: record.total_classes ||
0,
                        classesAttended:
record.classes_attended || 0,
                        attendancePercentage:
record.total_classes > 0
?
Math.round((record.classes_attended / record.total_classes) * 100)
: 0
                    };
                });

                // Calculate overall attendance percentage
                const overallAttendancePercentage =
totalClassesAll > 0
? Math.round((classesAttendedAll /
totalClassesAll) * 100)
: 0;

                resolve({
                    candidateDetails: {
                        id: candidate.id,
                        dateOfJoining:
candidate.dateOfJoining,
                        dateOfSparing:
candidate.dateOfSparing
                    },
                    monthlyRecords: formattedRecords,

```



```

        statistics: {
            totalMonths: totalRecords,
            totalClasses: totalClassesAll,
            classesAttended: classesAttendedAll,
            attendancePercentage:
overallAttendancePercentage
        }
    });
}
);
}
);
});
}

// Generate months between joining and sparing dates
async generateMonthsBetweenDates(candidateId) {
    return new Promise((resolve, reject) => {
        // Get candidate's joining and sparing dates
        db.get(
            'SELECT id, ticket_no,
date_of_joining_stc_wtc_non_railway as dateOfJoining, date_of_sparing as
dateOfSparing FROM wtc_candidates WHERE id = ?',
            [candidateId],
            (err, candidate) => {
                if (err) return reject(err);
                if (!candidate) return reject(new Error('Candidate
not found'));

                // If dates are not set, return empty array
                if (!candidate.dateOfJoining) {
                    return resolve({
                        candidateId: candidate.id,
                        ticketNo: candidate.ticket_no,
                        months: []
                    });
                }

                // Parse dates
                const startDate = new Date(candidate.dateOfJoining);
                // If sparing date is not set, use current date
                const endDate = candidate.dateOfSparing ? new
Date(candidate.dateOfSparing) : new Date();

                // Generate array of months between dates
                const months = [];
                let currentDate = new Date(startDate.getFullYear(),
startDate.getMonth(), 1);

                while (currentDate <= endDate) {
                    const month = currentDate.getFullYear() + '-' +
String(currentDate.getMonth() + 1).padStart(2, '0');
                    months.push(month);
                    currentDate.setMonth(currentDate.getMonth() + 1);
                }

                resolve({

```

```

        candidateId: candidate.id,
        ticketNo: candidate.ticket_no,
        months
    });
    }
    );
    });
}

// Bulk mark attendance for multiple candidates
async bulkMarkMonthlyAttendance(records) {
    const tableName = this.tableName; // Store table name in local
variable
    return new Promise((resolve, reject) => {
        const results = [];
        let processed = 0;

        if (records.length === 0) {
            return resolve({ success: true, results: [] });
        }

        // Begin transaction
        db.serialize(() => {
            db.run('BEGIN TRANSACTION');

            records.forEach(record => {
                const { candidateId, month, totalClasses,
classesAttended } = record;

                // Skip if required fields are missing
                if (!candidateId || !month || (totalClasses ===
undefined && classesAttended === undefined)) {
                    results.push({
                        candidateId,
                        month,
                        success: false,
                        message: 'Missing required fields'
                    });
                    processed++;

                    if (processed === records.length) {
                        db.run('COMMIT', err => {
                            if (err) {
                                db.run('ROLLBACK');
                                return reject(err);
                            }
                            resolve({ results });
                        });
                    }
                }
            });
            return;
        });

        // Check if candidate exists
        db.get(
            'SELECT id, ticket_no FROM wtc_candidates WHERE
id = ?',
            [candidateId],

```

```

(err, candidate) => {
  if (err || !candidate) {
    results.push({
      candidateId,
      month,
      success: false,
      message: err ? err.message :
'Candidate not found'
    });
    processed++;

    if (processed === records.length) {
      db.run('COMMIT', err => {
        if (err) {
          db.run('ROLLBACK');
          return reject(err);
        }
        resolve({ results });
      });
    }
    return;
  }

  // Check if a record already exists for this
  candidate and month
  db.get(
    `SELECT id FROM ${tableName} WHERE
candidate_id = ? AND month = ?`,
    [candidateId, month],
    (err, existing) => {
      if (err) {
        results.push({
          candidateId,
          month,
          success: false,
          message: err.message
        });
        processed++;

        if (processed === records.length)
{
          db.run('COMMIT', err => {
            if (err) {
              db.run('ROLLBACK');
              return reject(err);
            }
            resolve({ results });
          });
        }
        return;
      }

      if (existing) {
        // Update existing record
        const updateFields = [];
        const updateValues = [];

```

```

totalClasses >= 0) {
    if (totalClasses !== undefined &&
updateFields.push('total_classes = ?');
updateValues.push(totalClasses);
    }

    if (classesAttended !== undefined
&& classesAttended >= 0) {
updateFields.push('classes_attended = ?');
updateValues.push(classesAttended);
    }

    updateFields.push('updated_at =
CURRENT_TIMESTAMP');
    updateValues.push(existing.id);
    db.run(
        `UPDATE ${tableName} SET
        [...updateValues],
        function (err) {
            results.push({
                candidateId,
                month,
                success: !err,
                message: err ?
err.message : 'Attendance updated successfully'
            });
            processed++;
            if (processed ===
records.length) {
                db.run('COMMIT', err
                    if (err) {
                        return
                    }
                    resolve({ results
                });
            }
        }
    );
} else {
    // Insert new record
    db.run(
        `INSERT INTO ${tableName}
        (candidate_id, ticket_no,
month, total_classes, classes_attended)
        VALUES (?, ?, ?, ?, ?)`;

```



```

        whereClause += batch ? ' AND c.module_no = ?' : 'WHERE
c.module_no = ?';
        params.push(moduleNo);
    }

    // First get all candidates
    const candidateQuery = `
        SELECT
            c.id as candidateId,
            c.ticket_no as ticketNo,
            c.name,
            c.designation,
            c.batch,
            c.module_no as moduleNo,
            c.date_of_joining_stc_wtc_non_railway as
dateOfJoining,
            c.date_of_sparing as dateOfSparing
        FROM
            wtc_candidates c
        ${whereClause}
    `;

    db.all(candidateQuery, params, async (err, candidates) => {
        if (err) return reject(err);

        // For each candidate, get their monthly attendance
        const candidatesWithAttendance = [];

        // Process each candidate sequentially with Promise.all
        await Promise.all(candidates.map(async (candidate) => {
            try {
                // Get monthly attendance for this candidate
                const monthlyAttendance = await
this.getMonthlyAttendanceByCandidate(candidate.candidateId);

                candidatesWithAttendance.push({
                    ...candidate,
                    monthlyRecords:
monthlyAttendance.monthlyRecords,
                    statistics: monthlyAttendance.statistics
                });
            } catch (error) {
                // If there's an error, add the candidate without
attendance data
                candidatesWithAttendance.push({
                    ...candidate,
                    monthlyRecords: [],
                    statistics: {
                        totalMonths: 0,
                        totalClasses: 0,
                        classesAttended: 0,
                        attendancePercentage: 0
                    }
                });
            }
        }));
    });
}
    ));
}

```

```

        // Calculate overall statistics
        let totalClasses = 0;
        let totalClassesAttended = 0;

        candidatesWithAttendance.forEach(candidate => {
            totalClasses += candidate.statistics?.totalClasses ||
0;
            totalClassesAttended +=
candidate.statistics?.classesAttended || 0;
        });

        const averageAttendancePercentage = totalClasses > 0
            ? Math.round((totalClassesAttended / totalClasses) *
100)
            : 0;

        resolve({
            candidates: candidatesWithAttendance,
            summary: {
                totalCandidates: candidatesWithAttendance.length,
                totalClasses,
                totalClassesAttended,
                averageAttendancePercentage
            }
        });
    });
}

module.exports = new MonthlyAttendanceModel();

```

Key Features:

- Creates and manages the `monthly_attendances` table with unique constraints and foreign key references.
- Marks or updates monthly attendance for individual candidates, with automatic handling of new or existing records.
- Retrieves all monthly attendance records and statistics for a specific candidate.
- Generates a list of months between a candidate's joining and sparing dates for attendance tracking.
- Supports bulk marking of attendance for multiple candidates in a single transaction, with per-record validation and error handling.
- Provides summary statistics of attendance for all candidates, with optional filtering by batch or module.
- Ensures data integrity and consistency with robust error handling and validation throughout all operations.

3.8.4 Candidate Data Models

The files define the data models for managing candidate records in the Non-Railway, STC, and WTC modules, respectively. Each model is responsible for creating and maintaining its own database table, and provides a comprehensive set of methods for CRUD operations, filtering, and data retrieval. The models ensure data integrity through unique constraints and support both modern and legacy API patterns. The WTC model additionally includes utilities for converting between camelCase and snake_case to maintain consistency between JavaScript and database field naming conventions.

Key Features:

- **Table Creation and Schema Management:** Each model automatically creates its respective table with all necessary fields for personal, contact, professional, and course information.

```
db.run(`CREATE TABLE IF NOT EXISTS ${this.tableName} (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  // ... various fields ...
  ticket_no TEXT UNIQUE NOT NULL,
  // ... more fields ...
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  updated_at DATETIME DEFAULT CURRENT_TIMESTAMP
)`);
```

- **Create Candidate:** Inserts a new candidate record into the database, handling unique constraints on ticket numbers.

```
create(candidateData) {
  return new Promise((resolve, reject) => {
    const sql = `INSERT INTO ${this.tableName} (...fields...)
VALUES (...placeholders...)`;
    db.run(sql, [...values], function(err) {
      if (err) {
        if (err.message.includes('UNIQUE constraint
failed')) {
          reject(new Error('Ticket number already
exists'));
        } else {
          reject(err);
        }
      } else {
        resolve({ id: this.lastID, ticket_no:
candidateData.ticket_no, ...candidateData });
      }
    });
  });
}
```


- **Retrieve Candidate by Ticket Number:** Fetches a candidate's full record using their unique ticket number.

```
getByTicketNumber(ticketNumber) {
  return new Promise((resolve, reject) => {
    const sql = `SELECT * FROM ${this.tableName} WHERE
ticket_no = ?`;
    db.get(sql, [ticketNumber], (err, row) => {
      if (err) reject(err);
      else resolve(row);
    });
  });
}
```

- **Update Candidate by Ticket Number:** Updates all relevant fields for a candidate, including automatic timestamp updates.

```
updateByTicketNumber(ticketNumber, candidateData) {
  return new Promise((resolve, reject) => {
    const sql = `UPDATE ${this.tableName} SET ...fields...,
updated_at = CURRENT_TIMESTAMP WHERE ticket_no = ?`;
    db.run(sql, [...values, ticketNumber], function(err) {
      if (err) reject(err);
      else resolve(this.changes > 0 ? { ticket_no:
ticketNumber, ...candidateData } : null);
    });
  });
}
```

- **Delete Candidate by Ticket Number:** Removes a candidate record from the database using their ticket number.

```
deleteByTicketNumber(ticketNumber) {
  return new Promise((resolve, reject) => {
    const sql = `DELETE FROM ${this.tableName} WHERE ticket_no
= ?`;
    db.run(sql, [ticketNumber], function(err) {
      if (err) reject(err);
      else resolve(this.changes > 0);
    });
  });
}
```

- **Filtering and Specialized Queries:** Provides methods to filter candidates by designation, unit, course type, batch, or module number.

```
getByDesignation(designation) {
  return new Promise((resolve, reject) => {
    const sql = `SELECT * FROM ${this.tableName} WHERE
designation = ?`;
    db.all(sql, [designation], (err, rows) => {
      if (err) reject(err);
      else resolve(rows);
    });
  });
}
```

```

    });
  });
}

```

- **Backward Compatibility:** Methods to support legacy API patterns using candidate IDs.
- **Naming Convention Utilities (WTC):** Converts between camelCase (JavaScript) and snake_case (database) for seamless data handling.

```

// Converts a camelCase string to snake_case
const toSnakeCase = (str) => str.replace(/[A-Z]/g, letter =>
`_${letter.toLowerCase()}`);
// Converts all keys of an object to snake_case
const convertToSnakeCase = (obj) => { ... };
// Converts all keys of an object to camelCase
const convertToCamelCase = (obj) => { ... };

```

3.8.5 Course Score Models

The course score models define the data layer for managing detailed session-wise and paper-wise marks for candidates in various technical and practical training modules in each of the 12 courses. Each model creates and manages its own table, supports CRUD operations, paper-specific updates, marks summaries, and advanced queries that join candidate and score data. The models ensure data integrity with unique constraints and foreign key relationships to the main candidate tables.

Key Features:

- **Table Creation and Schema Management:** Each model creates a table with fields for ticket number, session-wise and paper-wise marks, maximum marks, and system fields (timestamps).

```

db.run(`CREATE TABLE IF NOT EXISTS ${this.tableName} (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  ticket_no TEXT UNIQUE NOT NULL,
  // ... session and paper marks fields ...
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  updated_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY(ticket_no) REFERENCES stc_candidates(ticket_no)
)`);

```

- **Create Score Record:** Inserts a new score record for a candidate, handling unique constraints on ticket numbers.

```

create(scoreData) {
  return new Promise((resolve, reject) => {

```

```

        const sql = `INSERT INTO ${this.tableName} (...fields...)
VALUES (...placeholders...)`;
        db.run(sql, [...values], function(err) {
            if (err) {
                if (err.message.includes('UNIQUE constraint
failed')) {
                    reject(new Error('Ticket number already
exists'));
                } else {
                    reject(err);
                }
            } else {
                resolve({ id: this.lastID, ticket_no:
scoreData.ticket_no, ...scoreData });
            }
        });
    });
}

```

- **Retrieve Score by Ticket Number:** Fetches a candidate's full score record using their unique ticket number.

```

getByTicketNumber(ticketNumber) {
    return new Promise((resolve, reject) => {
        const sql = `SELECT * FROM ${this.tableName} WHERE
ticket_no = ?`;
        db.get(sql, [ticketNumber], (err, row) => {
            if (err) reject(err);
            else resolve(row);
        });
    });
}

```

- **Update Score by Ticket Number:** Updates all relevant marks fields for a candidate, including automatic timestamp updates.

```

updateByTicketNumber(ticketNumber, scoreData) {
    return new Promise((resolve, reject) => {
        const sql = `UPDATE ${this.tableName} SET ...fields...,
updated_at = CURRENT_TIMESTAMP WHERE ticket_no = ?`;
        db.run(sql, [...values, ticketNumber], function(err) {
            if (err) reject(err);
            else resolve(this.changes > 0 ? { ticket_no:
ticketNumber, ...scoreData } : null);
        });
    });
}

```

- **Delete Score by Ticket Number:** Removes a score record from the database using the ticket number.
- **Update Specific Paper Marks:** Allows updating marks for a specific paper within a candidate's score record.

```

updatePaperMarks(ticketNumber, paperCode, marks) {
  return new Promise((resolve, reject) => {
    const sql = `UPDATE ${this.tableName} SET
    ${paperCode}_marks = ?, updated_at = CURRENT_TIMESTAMP WHERE
    ticket_no = ?`;
    db.run(sql, [marks, ticketNumber], function(err) {
      if (err) reject(err);
      else resolve(this.changes > 0);
    });
  });
}

```

- **Get Marks Summary:** Retrieves a summary of total marks and maximum possible marks for a candidate.

```

getMarksSummary(ticketNumber) {
  return new Promise((resolve, reject) => {
    const sql = `SELECT *, (slp1_marks + slp2_marks + ...) as
    total_marks, (slp1_max_marks + slp2_max_marks + ...) as
    total_max_marks FROM ${this.tableName} WHERE ticket_no = ?`;
    db.get(sql, [ticketNumber], (err, row) => {
      if (err) reject(err);
      else resolve(row);
    });
  });
}

```

- **Scores with Candidate Details:** Provides methods to join and fetch both candidate and score details for richer reporting.

```

getScoresWithCandidateDetails() {
  return new Promise((resolve, reject) => {
    const sql = `
    SELECT sc.*, mji.slp1_marks, mji.slp2_marks, ...,
    (mji.slp1_marks + ...) as total_marks
    FROM stc_candidates sc
    LEFT JOIN ${this.tableName} mji ON sc.ticket_no =
    mji.ticket_no
    ORDER BY sc.created_at DESC
    `;
    db.all(sql, [], (err, rows) => {
      if (err) reject(err);
      else resolve(rows);
    });
  });
}

```

- **Backward Compatibility:** Methods to support legacy API patterns using candidate IDs.

3.9 COMPLETE AUTHENTICATION FLOW

3.9.1 Authentication Architecture Overview

The STVT backend implements a session-based authentication system with the following components:

```
[Client] → [Routes] → [Controller] → [User Model] → [Session Utils] →  
[Database]  
↓  
[Auth Middleware] → [Session Validation] → [Route Protection]
```

3.9.2 Authentication Flow Diagram

```
1. User Login Request  
↓  
2. authController.login()  
↓  
3. Input Validation (role, password)  
↓  
4. User.verify() → SQLite Database  
↓  
5. createSession() → In-Memory Storage  
↓  
6. Return Token to Client  
↓  
7. Client Stores Token  
↓  
8. Subsequent Requests with Authorization Header  
↓  
9. authenticateToken Middleware  
↓  
10. Session Validation & Extension  
↓  
11. Route Access Granted
```

3.9.3 Training Module Architecture

The system supports multiple training types with a consistent structure:

Course-Specific Routes (12 Total):

- MSE (Mechanical Signal Equipment): C&W, D, W
- MJR (Major): C&W, D, W
- MJI (Major Infrastructure): C&W, D, W
- MJP (Major Project): C&W, D, W

Core Training Routes:

- STC (Supervisor's Training Centre)
- WTC (Workshop Training Centre)
- Non-Railway Training
- Line Training (For STC)
- Monthly Attendance

Common Pattern:

```
// Each training type follows this pattern:
app.use("/api/[training-type]", [training-type]Routes);

// Routes → Controllers → Models → Database
```

3.10 BACKEND ARCHITECTURE SUMMARY

3.10.1 Technology Stack

- **Runtime:** Node.js
- **Framework:** Express.js
- **Database:** SQLite3
- **Authentication:** Session-based (in-memory)
- **File Storage:** Local filesystem

3.10.2 Request Flow

1. **HTTP Request** → Express Router
2. **Router** → Authentication Middleware (if protected)
3. **Middleware** → Controller Function
4. **Controller** → User Model / Database Operations
5. **Database** → SQLite Operations
6. **Response** flows back through the chain

3.10.3 Key Design Patterns

- **MVC Pattern:** Clear separation of routes, controllers, and models
- **Middleware Pattern:** Reusable request processing (authentication, CORS)
- **Module Pattern:** Each feature (training type) has its own routes/controllers
- **Session Pattern:** In-memory session storage with automatic cleanup

3.10.4 Security Features

- **Role-Based Access Control:** Admin, Master, Operator roles
- **Session Management:** Automatic session expiration and cleanup
- **Input Validation:** Request validation in controllers
- **CORS Configuration:** Cross-origin request handling
- **Error Handling:** Comprehensive error responses

3.10.5 File Structure Benefits

- **Modular Design:** Each training type has dedicated routes/controllers
- **Scalability:** Easy to add new training types
- **Maintainability:** Clear separation of concerns
- **Code Reusability:** Common patterns across different modules

3.10.6 Data Flow Example

```
// Example: User Login Flow
POST /api/auth/login
  → authRoutes.js (router.post('/login', login))
  → authController.js (login function)
  → User.verify(role, password)
  → SQLite database query
  → createSession(user)
  → Return token to client
```

3.10.7 Error Handling Strategy

- **Input Validation:** 400 Bad Request for invalid inputs
- **Authentication:** 401 Unauthorized for missing tokens
- **Authorization:** 403 Forbidden for expired sessions
- **Server Errors:** 500 Internal Server Error with logging
- **Not Found:** 404 for invalid routes

CHAPTER 4

FRONTEND CODE ARCHITECTURE AND IMPLEMENTATION

4.1 INTRODUCTION

The frontend of the project serves as the primary interface through which users interact with the system. Built using modern web technologies, the frontend is designed to provide a seamless, responsive, and user-friendly experience for a diverse set of users, including administrators, trainees, and management staff.

This implementation leverages the power of React for building dynamic user interfaces, with Vite as the build tool to ensure fast development and optimized production builds. The project structure is organized for scalability and maintainability, with clear separation of concerns across components, pages, and utility modules.

Key features of the frontend include:

- **Marksheet Management:** Allowing users to view, generate, and manage marksheets for various training programs.
- **Certificate Generation:** Providing dynamic certificate creation and preview functionalities for successful trainees.
- **Letter Issuance:** Enabling the generation of official letters, such as interview and training letters, tailored to individual trainees.

The frontend communicates with the backend via RESTful APIs, ensuring real-time data synchronization and secure access to resources. Special attention has been given to user experience, accessibility, and consistent design throughout the application.

This chapter details the architectural choices, component structure, and implementation strategies adopted in the frontend, with a particular focus on the marksheet, certificate, and letter features.

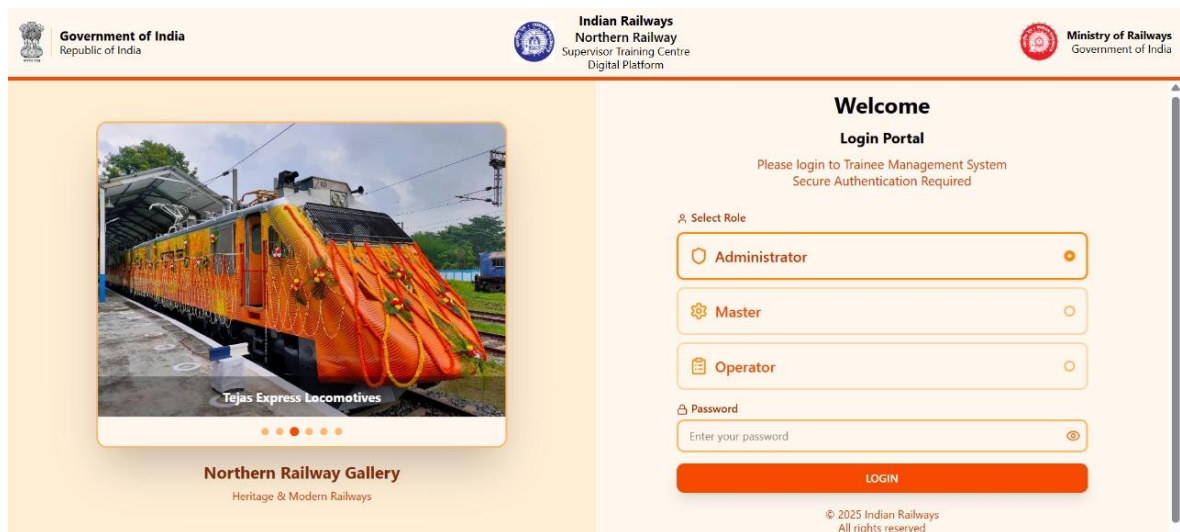


Fig. 4.1 Login Page

4.2 PROJECT STRUCTURE

The frontend of the project is carefully organized to ensure clarity, modularity, and ease of maintenance. The structure follows best practices for modern React applications, making it straightforward for developers to navigate and extend the codebase.

At the top level, the frontend directory contains essential configuration files, static assets, and the main source code folder. This organization separates concerns and allows for efficient development and deployment.

4.2.1 Top-Level Files and Folders

The root of the frontend directory includes:

- **package.json / package-lock.json:** Define project dependencies, scripts, and metadata for package management.
- **vite.config.js:** Configuration for the Vite build tool, enabling fast development and optimized builds.
- **index.html:** The main HTML entry point for the application, loaded by Vite.
- **public/:** Contains static assets such as images and icons that are publicly accessible.
- **src/:** The source directory containing all application logic, components, and styles.

This separation ensures that configuration, static resources, and source code are logically grouped, simplifying both development and deployment processes.

4.2.2 Source Directory

The `src/` folder is the core of the frontend implementation. It is structured to support feature-based development and code reuse.

Key files in `src/`:

- **App.jsx**: The root React component that sets up the main application layout and routing.
- **main.jsx**: The entry point for rendering the React application.
- **index.css**: Global CSS styles applied throughout the application.
- **Router.jsx**: Defines the client-side routing logic, mapping URLs to components.

Feature and domain-specific folders:

- **auth/**
 - Handles authentication logic, including:
 - **AuthContext.jsx**: Provides authentication state and methods via React Context.
 - **Login.jsx**: The login form and logic for user authentication.
- **components/**
 - Contains reusable UI components such as the dashboard and navigation bar.
- **manage/**
 - Implements candidate management features, including:
 - **CandidateManagementPage.jsx**, **CandidateTable.jsx**, **DetailModal.jsx**, etc.
- **stcManagement/**
 - Manages STC (Special Training Course) related features, including:
 - **Marksheet.jsx**: Marksheet generation and display.
 - **Certificate.jsx**, **CertificatePreview.jsx**, **CertificatePreview2.jsx**: Certificate creation and preview.
 - Other components for feedback, trainee profiles, and line training.
- **WTCManagement/**
 - Manages WTC (Workshop Training Course) related features, including:
 - **Certificate.jsx**, **CertificatePreview.jsx**: Certificate handling for WTC.
 - **AttendanceSystem.jsx**, **HomePageWTC.jsx**, **TraineeProfile.jsx**: WTC-specific management.
 - Letter generation and preview components such as **Letter.jsx**, **LetterPreview.jsx**, **PracticalExamInterviewLetter.jsx**, **PracticalTrainingLetter.jsx**, and **SessionalPerformanceLetter.jsx**.

- **NonRailway/, STC/, WTC/**
 - Contain form components for data entry related to different training programs, organized into subfolders like form/ for modularity.
- **assets/**
 - Stores images, fonts, and other static resources used in the UI.

This modular structure allows for:

- **Separation of concerns:** Logic for authentication, management, and feature-specific tasks are separated into distinct folders.
- **Reusability:** Common components are placed in shared folders for reuse across the application.
- **Scalability:** Features are grouped by domain, making it easy to locate and update related code as the project grows.

By following this structure, the frontend remains maintainable and scalable, supporting both current requirements and future enhancements.

4.3 STATE MANAGEMENT AND ROUTING

State management in the frontend is primarily handled using React's built-in state mechanisms, with additional use of the Context API for managing global state such as user authentication. The `auth/AuthContext.jsx` file defines a context provider that encapsulates authentication logic, making the current user's authentication status and related methods available throughout the component tree. This approach ensures that components at any level can access and update authentication state without the need for prop drilling, thereby improving code maintainability and scalability.

For local component state, React's `useState` and `useEffect` hooks are extensively used. These hooks manage form inputs, loading states, and other UI-related data within individual components. In more complex scenarios, such as managing candidate data or marksheet entries, state is often lifted to parent components or managed through context to facilitate data sharing between related components.

Routing within the application is managed using React Router, as defined in the `Router.jsx` file. This setup enables client-side navigation between different pages and features without requiring full page reloads. Routes are mapped to specific components, such as dashboards, management pages, and forms for marksheet, certificate, and letter generation. Protected routes are implemented to restrict access to certain pages based on the user's authentication status, leveraging the authentication context for access control.

The combination of React's state management and React Router provides a seamless and responsive user experience. Users can navigate between different sections of the application, such as viewing marksheets, generating certificates, or managing candidates, with state persisting appropriately across navigation events. This architecture supports both the modularity and interactivity required for a modern, feature-rich web application.

4.4 MARKSHEET IMPLEMENTATION

The marksheet feature is a core component of the application, enabling users to generate, view, and manage marksheets for various training programs. This section details the design, component structure, data flow, and user interface considerations for the marksheet functionality.

4.4.1 Overview

The marksheet system is designed to allow authorized users to review, and export marks for trainees. It supports both the display of individual session results and the aggregation of performance data for the course duration. The feature is primarily accessed by administrators and Coordinators.

In cases where a trainee has cleared a supplementary examination, the marksheet reflects a "Passed" status along with the supplementary marks, but the original score is used for overall percentage calculations.

4.4.2 Component Structure

The marksheet functionality is implemented using modular React components, primarily located in the `stcManagement/` directory.

Key components:

- **Marksheet.jsx:** Handles the main logic for displaying and managing marksheets.
- Supporting components such as `StatsCard.jsx` and `ResultSummary.jsx` are used to present summary statistics and detailed results.

This modular approach allows for easy maintenance and extension of the marksheet feature.

4.4.3 Data Flow

Data for the marksheet is typically fetched from the backend via RESTful API calls. Upon loading the marksheet page, the component sends a request to retrieve trainee data and their corresponding marks. The data is then stored in the component's state and rendered in a tabular format.

Example data fetching logic:

```
useEffect(() => {
  // Only fetch data if a candidate is selected and courseCode is
  // available
  if (candidateData && courseCode) {
    const fetchMarksheetData = async () => {
      setLoading(true);
      try {
        // API call to fetch marksheet data for the selected candidate
        // and course
        const marks = await marksheetService.getMarksheetData(
          candidateData.ticketNumber,
          courseCode
        );
        if (marks.data === null) {
          setMarksheetData(null);
          setFailedSubjects([]);
          setMessage({
            type: "info",
            text: `No marks added yet for ${candidateData.name}.`,
          });
        } else {
          setMarksheetData(marks.data);
          // Optionally fetch failed subjects
          const failed = await marksheetService.getFailedSubjects(
            candidateData.ticketNumber,
            courseCode,
            marks.data
          );
          setFailedSubjects(failed.data);
          setMessage({
            type: "success",
            text: `Marksheet loaded for: ${candidateData.name}`,
          });
        }
      } catch (error) {
        setMessage({
          type: "error",
          text: error.message || "Failed to load marksheet data",
        });
        setMarksheetData({});
        setFailedSubjects([]);
      } finally {
        setLoading(false);
      }
    };
  }
};
```

```

        fetchMarksheetData();
    }
}, [candidateData, courseCode]);

```

User interactions, such as editing marks or adding new entries, update the local state and can trigger API requests to persist changes to the backend.

4.4.4 UI/UX Considerations

The marksheet interface is designed for clarity and ease of use. Key considerations include:

- Displaying marks in a well-structured table with clear headings and row highlights.
- Providing input validation to prevent erroneous data entry.
- Offering option to print the marksheet for distribution.

```

const printContent = marksheetRef.current.cloneNode(true);

// Apply print-specific styles to ensure consistent output
printContent.style.cssText = `
background: white !important;
color: black !important;
font-family: 'Times New Roman', serif !important;
font-size: 14px !important;
line-height: 1.4 !important;
width: 100% !important;
padding: 20px !important;
margin: 0 !important;
box-sizing: border-box !important;
box-shadow: none !important;
border-radius: 0 !important;
border: none !important;
`;

printWindow.document.write(`
<!DOCTYPE html>
<html>
<head>
<title>Marksheet - ${candidateData.name}
(${candidateData.ticketNumber || candidateData.ticket_no})</title>
<style>
@page {
    size: A4;
    margin: 8mm;
}
@media print {
    * {
        -webkit-print-color-adjust: exact !important;
        color-adjust: exact !important;
        print-color-adjust: exact !important;

```

```

    }
    body {
        margin: 0 !important;
        padding: 0 !important;
        background: white !important;
        font-family: 'Times New Roman', serif !important;
        font-size: 14px !important;
        line-height: 1.4 !important;
        color: black !important;
    }
    .no-print {
        display: none !important;
    }
    table {
        border-collapse: collapse !important;
        page-break-inside: auto !important;
    }
    tr {
        page-break-inside: avoid !important;
        page-break-after: auto !important;
    }
    thead {
        display: table-header-group !important;
    }
    tfoot {
        display: table-footer-group !important;
    }
}
body {
    margin: 0;
    padding: 0;
    background: white;
    font-family: 'Times New Roman', serif;
    font-size: 14px;
    line-height: 1.4;
    color: black;
}
</style>
</head>
<body>
    ${printContent.outerHTML}
</body>
</html>
`);

printWindow.document.close();
printWindow.focus();

// Wait for images to load, then print
setTimeout(() => {
    printWindow.print();
    printWindow.close();
}, 1000);

```

Error messages and loading indicators are used to enhance the user experience and provide feedback during data operations.

[illegible]

Fig. 4.2 Marksheet Preview

4.5 CERTIFICATE IMPLEMENTATION

The certificate feature enables the generation, preview, and export of training certificates for successful trainees. This section describes the design, component structure, data flow, and user interface considerations for certificate handling in the frontend.

4.5.1 Overview

The certificate system is designed to automate the creation of official certificates for trainees who have completed their respective courses. It ensures that all certificates are generated with consistent formatting, accurate trainee data, and are ready for printing or digital distribution.

The certificate preview includes editable sections, allowing users to modify specific details before finalizing the print, ensuring flexibility and accuracy in the issued certificates.

4.5.2 Component Structure

Certificate-related functionality is implemented using dedicated React components, organized by training program:

- STC Certificates:
 - stcManagement/Certificate.jsx
 - stcManagement/CertificatePreview.jsx

- stcManagement/CertificatePreview2.jsx
- WTC Certificates:
 - WTCManagement/Certificate.jsx
 - WTCManagement/CertificatePreview.jsx

These components are responsible for:

- Collecting trainee and course data.
- Rendering the certificate layout with dynamic data.
- Providing a live preview before export or print.

Main Structure of a Certificate Preview:

```
const CertificateTemplate = ({ trainee }) => {
  const defaultDuration = courseStructure[trainee.module_no] || "";
  const [editableDuration, setEditableDuration] =
    useState(defaultDuration);
  const [editableTicketNo, setEditableTicketNo] =
    useState(trainee.ticket_no);

  useEffect(() => {
    setEditableDuration(defaultDuration);
    setEditableTicketNo(trainee.ticket_no);
  }, [trainee]);

  return (
    <div className="certificate-preview-outer">
      <div
        className="certificate border-double border-4 border-black bg-
white relative overflow-hidden shadow-xl"
        style={{
          fontFamily: "Times New Roman, serif",
          width: "950px",
          minHeight: "670px",
          margin: "4px auto",
          background: "#fff",
          borderRadius: "18px",
          boxShadow: "0 8px 32px 0 rgba(31, 38, 135, 0.18)",
          position: "relative",
          padding: "0",
          display: "flex",
          flexDirection: "column",
          justifyContent: "space-between",
        }}
      >
        { /* Watermark */ }
        <img
          src={BgImage}
          alt="Watermark"
          style={{
            position: "absolute",
            top: "50%",
            left: "50%",

```

```

        width: "92%",
        height: "92%",
        transform: "translate(-50%, -50%)",
        opacity: 0.07,
        zIndex: 0,
        pointerEvents: "none",
        objectFit: "contain",
        filter: "blur(0.5px)",
    }}
    draggable={false}
  />
  { /* Header */ }
  <div className="relative z-10" style={{ padding: "38px 60px 40px
60px" }}>
    <div className="flex justify-between items-start">
      <div>
        <img
          src={BgImage}
          alt="Logo"
          style={{
            width: 100,
            filter: "drop-shadow(0 2px 6px #aaa)",
          }}
        />
      </div>
      <div className="text-center flex-1" style={{ marginLeft: "-
25px" }}>
        <div
          style={{
            fontSize: 36,
            fontWeight: 700,
            color: "#17408B",
            marginBottom: 0,
            letterSpacing: "1px",
          }}
        >
          Supervisors Training Centre
        </div>
        <div
          style={{
            fontSize: 22,
            fontWeight: 600,
            color: "#17408B",
            marginBottom: 0,
          }}
        >
          Northern Railway, Charbagh, Lucknow
        </div>
        <div
          style={{
            fontSize: 36,
            fontWeight: 700,
            color: "#17408B",
            margin: "18px 0 0 0",
            letterSpacing: "2px",
          }}
        >

```

```

        Certificate
      </div>
    </div>
  </div>
</div>
  { /* ...rest of the certificate content (trainee details, course
info, signatures, etc.)... */}
</div>
</div>
);
};

```

4.5.3 Certificate Generation Workflow

The workflow for certificate generation typically involves:

1. **Data Entry:** The user selects a trainee or enters relevant details.
2. **Preview:** The certificate is rendered in real-time using the provided data.
3. **Export/Print:** The user can export the certificate as a PDF or print it directly from the browser.

Passing Data to the Preview Component:

```

const generateBulkCertificates = () => {
  if (selectedTrainees.length === 0) {
    alert("Please select at least one trainee");
    return;
  }
  const selectedTraineeObjects = trainees.filter((trainee) =>
    selectedTrainees.includes(trainee.ticket_no)
  );

  try {
    sessionStorage.setItem(
      "selectedTrainees",
      JSON.stringify(selectedTraineeObjects)
    );
    const traineesParam = encodeURIComponent(
      JSON.stringify(selectedTrainees)
    );
    // Choose preview component based on trainee/module type
    const isRegular = selectedTraineeObjects.every((trainee) => {
      const ticketNo = trainee.ticket_no ?
trainee.ticket_no.toUpperCase() : "";
      const moduleNo = trainee.module_no ?
trainee.module_no.toLowerCase() : "";
      return (
        (ticketNo.startsWith("ASE") ||
        ticketNo.startsWith("AJE") ||
        ticketNo.startsWith("IJE") ||
        ticketNo.startsWith("RSE")) &&

```

```

        moduleNo !== "other" &&
        moduleNo !== "custom"
    );
});

if (isRegular) {
    navigate(`/stc/certificate/preview?trainees=${traineesParam}`);
} else {
    navigate(`/stc/certificate/preview2?trainees=${traineesParam}`);
}
} catch (error) {
    console.error("Error storing trainee data:", error);
    alert("Error preparing certificate data. Please try again.");
}
};

```

4.5.4. UI/UX Considerations

- **Consistent Layout:** All certificates follow a standardized template, ensuring uniformity across different batches and programs.
- **Live Preview:** Users can see exactly how the certificate will appear before finalizing.
- **Accessibility:** Font choices, contrast, and layout are optimized for readability and professional appearance.
- **Export Options:** Certificates can be exported as PDFs or printed, using similar techniques as the marksheet (e.g., using a ref and print-specific styles).



Fig. 4.3 STC Certificate Preview

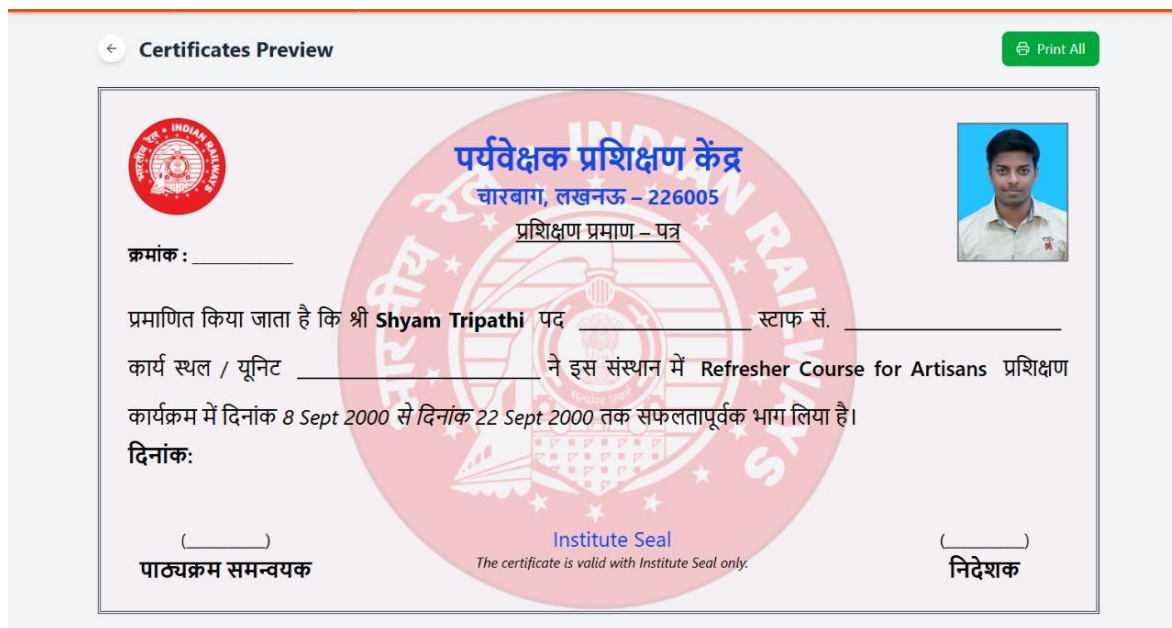


Fig. 4.4 WTC Certificate Preview

4.6 LETTER IMPLEMENTATION

The letter feature enables the generation, preview, and export of official letters such as Practical Exam Interview Letters, Practical Training Letters, and Sessional Performance Letters for trainees. This section describes the design, component structure, data flow, and user interface considerations for letter handling in the frontend.

4.6.1 Overview

The letter system is designed to automate the creation of various official letters required during the training process. It ensures that all letters are generated with consistent formatting, accurate trainee data, and are ready for printing or digital distribution.

4.6.2 Component Structure

Letter-related functionality is implemented using dedicated React components, organized by letter type and training program

WTC Letters:

- WTCManagement/Letter.jsx

- WTCManagement/LetterPreview.jsx
- WTCManagement/PracticalExamInterviewLetter.jsx
- WTCManagement/PracticalTrainingLetter.jsx
- WTCManagement/SessionalPerformanceLetter.jsx

These components are responsible for:

- Collecting trainee and letter-specific data.
- Rendering the letter layout with dynamic data.
- Providing a live preview before export or print.

Letter Preview:

```
const LetterPreview = () => {
  const [trainees, setTrainees] = useState([]);
  const [letterType, setLetterType] = useState('practical-training');

  // ...data loading logic...

  const LetterTemplate = ({ trainee }) => {
    // Render the appropriate letter template based on the selected type
    switch (letterType) {
      case 'practical-training':
        return <PracticalTrainingLetter trainee={trainee} />;
      case 'practical-exam':
        return <PracticalExamInterviewLetter trainee={trainee} />;
      case 'sessional-performance':
        return <SessionalPerformanceLetter trainee={trainee} />;
      default:
        return <PracticalTrainingLetter trainee={trainee} />;
    }
  };

  return (
    <div className="letter-preview-container">
      {trainees.map((trainee, idx) => (
        <div key={trainee.id || idx} className="mb-12 page-break-after">
          <LetterTemplate trainee={trainee} />
        </div>
      ))}
    </div>
  );
};
```

4.6.3 UI/UX Considerations

- **Consistent Layout:** All letters follow a standardized template, ensuring uniformity across different batches and programs.

- **Live Preview:** Users can see exactly how the letter will appear before finalizing and also edit details in some sections as per requirement.
- **Accessibility:** Font choices, contrast, and layout are optimized for readability and professional appearance.
- **Export Options:** Letters can be exported as PDFs or printed, using similar techniques as the marksheet and certificate (e.g., using a ref and print-specific styles).

NORTHERN RAILWAY
 Office of the Director
 Supervisors' Training Centre
 Northern Railway
 Charbagh, Lucknow – 226005

No. STC/CG/16

DATE: XXXXXXXX

XXXXXX
 XXXXXXXXXX
 XXXXX, Lucknow

Sub: Practical Training of C.G. Apprentices on Shop Floor/Job Training.

Undernoted CG Apprentices are directed to you for trade related on job Practical Training for the period mentioned against them. Please send their attendance to this office on 14th of every month during training and arrange to spare them after completion of the training period.

S.N.	NAME/ Sh./Km/Smt.	T.No.	TRADE	FROM	TO
1.	Joseph Carlos	S00001	XXXXX	XXXXXXXX	XXXXXXXX

This has approval of the competent authority.

For Director

Copy to: Ch. OS/Time Office/ STC for n/action please.

Fig. 4.5 Letter Preview

4.7 BEST PRACTICES, CHALLENGES, AND CONCLUSION

The frontend implementation of this project is guided by a set of best practices and design patterns that ensure maintainability, scalability, and a positive user experience. Throughout development, several challenges were encountered and addressed, leading to a robust and user-friendly application.

4.7.1 Best Practices and Patterns

- **Modular Component Design:** The application is structured around reusable and modular React components. Features such as marksheet, certificate, and letter generation are encapsulated in their own components, promoting code reuse and simplifying maintenance.
- **State Management:** State is managed using React's built-in hooks and the Context API for global state, such as authentication. This approach avoids prop drilling and keeps state logic close to where it is used.
- **Consistent Styling:** Styling is handled through a combination of global CSS and component-level classes, ensuring a consistent look and feel across the application. Print and export views are carefully styled to match on-screen previews.
- **Separation of Concerns:** The codebase is organized by feature and domain, with clear separation between authentication, management, and feature-specific logic. This makes the codebase easier to navigate and extend.
- **User Experience:** Features such as live previews, validation, and clear feedback messages enhance usability. Export and print functionalities are implemented to provide WYSIWYG (What You See Is What You Get) outputs.

4.7.2 Challenges and Solutions

- **Dynamic Data Rendering:** Rendering complex documents like marksheets, certificates, and letters with dynamic data required careful handling of state and conditional rendering. This was addressed by designing flexible component structures and using utility functions for data formatting.
- **Export and Print Consistency:** Ensuring that exported or printed documents matched the on-screen preview was a challenge, especially with browser differences. This was solved by cloning DOM nodes, applying print-specific styles, and using libraries like html2canvas and jsPDF where needed.
- **Data Synchronization:** Keeping the frontend in sync with backend data, especially when handling multiple trainees or bulk operations, required robust API integration and error handling. The use of async functions, loading states, and user feedback helped maintain reliability.
- **Scalability:** As the number of features grew, maintaining a clean and scalable codebase became essential. This was achieved through modularization, clear folder structure, and adherence to React best practices.

4.7.3 Conclusion

The frontend of this project successfully delivers a comprehensive and user-friendly interface for managing marksheets, certificates, and official letters. By adhering to best practices in component design, state management, and user experience, the application remains maintainable and scalable. The challenges encountered during development led to thoughtful solutions that improved the overall robustness of the system. Moving forward, this solid foundation will support future enhancements and the addition of new features with ease.

CHAPTER 5

TESTING RESULT

5.1 INTRODUCTION

This chapter provides a detailed analysis of the simulation and testing processes conducted on the various modules of the Training Management System. The application is developed to streamline the training workflows of STC (Supervisor's Training Centre), WTC (Workshop Training Centre), and Non-Railway candidates, replacing manual data management with a centralized, digital solution.

The primary objective of this phase was to ensure that the system functions reliably under different user scenarios, handles data consistently, and delivers the expected outputs without failure. Each module—ranging from registration and marks entry to certificate generation and attendance tracking—was tested to validate data integrity, user interaction, and responsiveness.

5.1.1 Testing Methodology

The testing process followed a modular and iterative approach, covering both manual testing and functional testing:

- **Unit Testing:** Each module and component (form submissions, table views, PDF generation) was tested in isolation.
- **Integration Testing:** The interaction between frontend and backend components was verified using a local proxy setup.
- **Validation Testing:** Form fields were tested for correctness, completeness, and error handling during user input.
- **Performance Testing:** Application responsiveness was tested with various data loads to ensure smooth operation.
- **UI/UX Testing:** All screens were evaluated across different resolutions to confirm responsiveness and visual consistency.

5.1.2 Module-Wise Test Coverage

Table 5.1 Project Module Testing

Module	Test Performed	Status
Trainee Registration	Form submission, validation, database entry	Passed
Marks Entry	Input validation	Passed
Feeding Record	Date-based entries, updates, listing	Passed
Attendance Management	Date-wise logs, trainee mapping	Passed
Certificate Generation	Auto-fill data, export as PDF, preview view	Passed
Record Filtering & Search	Search by batch, date, department	Passed
Navigation & UI Handling	Page redirection, visibility of options	Passed
Error Handling	Invalid data inputs, empty field alerts, missing records	Passed

5.2 ADD & MANAGE CANDIDATE FOR STC, WTC, NON-RAILWAY

This section outlines the candidate management modules developed for STC, WTC, and Non-Railway training categories. These modules enable administrators to add, edit, and manage trainee records efficiently through structured forms with built-in validations.

Each form captures key trainee information and ensures data accuracy before storing it in the centralized database. The system streamlines the registration process, reduces manual effort, and supports real-time updates across all training categories.

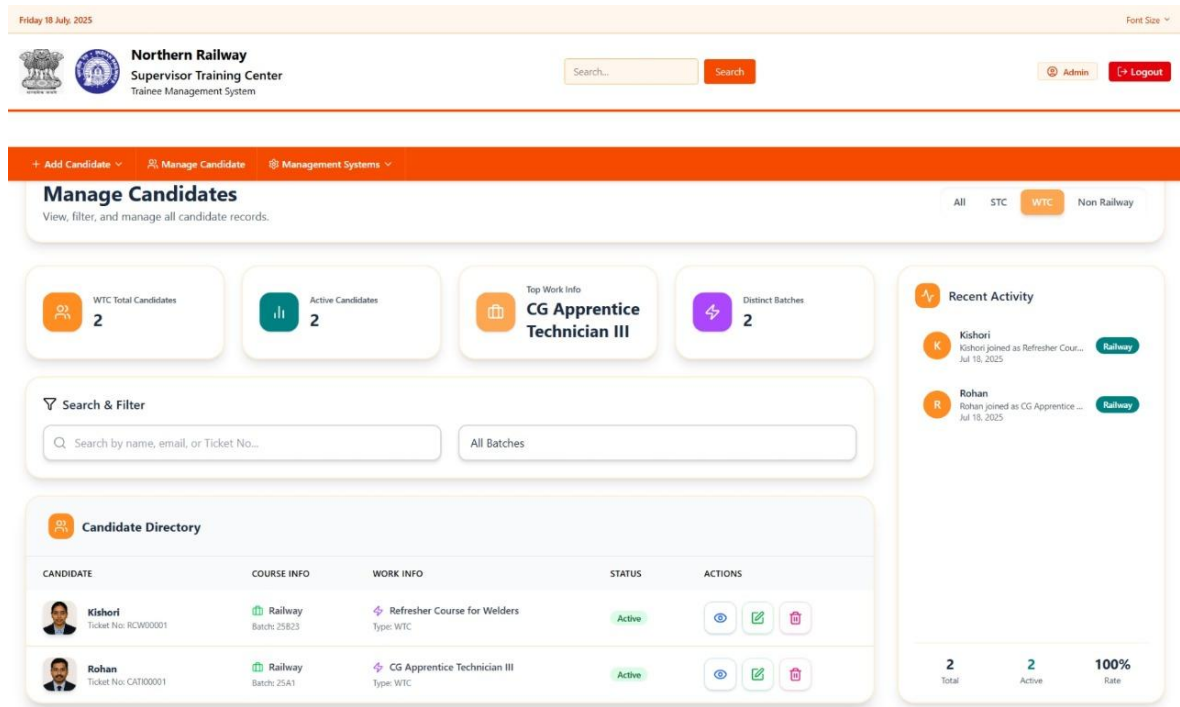


Fig. 5.1 Manage Candidate Page

5.2.1 STC Form

The STC Form Module is designed to allow administrators to efficiently input, manage, and validate trainee data specific to the Supervisor's Training Centre (STC). The form includes structured fields for critical information such as trainee name, ID, batch, and contact details.

Functionality and Validation

- Each field in the form is manually validated to ensure data completeness and correct formatting.
- Real-time alerts notify users of any missing or improperly formatted input before submission.
- Unique constraints (e.g., trainee ID) help prevent duplicate entries and maintain data integrity.

Testing Outcomes

- The module was tested with both valid and invalid data inputs.
- The system accurately flagged incomplete or incorrect entries, ensuring the robustness of manual validation.

- On successful submission, the data was immediately written to the SQLite3 database and reflected in the trainee dashboard.
- The form was tested across different screen sizes and devices to ensure responsive performance and consistent UI behavior.

Result

The STC form module performed as expected, offering a stable, user-friendly, and secure interface for candidate registration. It streamlines data entry and supports real-time monitoring and updates of STC trainee records.

STC Registration

Personal
Contact
Professional
Course

Personal Details

Picture *

Choose File
No file chosen

Name *

Enter full name

Gender *

Select gender
▼

Father's Name *

Enter father's name

Mother's Name

Enter mother's name

Date of Birth *

dd-mm-yyyy
📅

Category *

Select category
▼

PWD (Yes/No) *

Select
▼

Nationality *

Indian

Save and Next →

Fig. 5.2 STC Registration

5.2.2 WTC Form

The WTC Form Module is designed to manage trainee registration for the Workshop Training Centre (WTC). It enables administrators to input candidate data through a structured form that includes essential fields such as name, trainee ID, batch, and contact details.

Functionality and Validation

- Manual validation checks ensure that all required fields are completed correctly before submission.
- The system detects missing or incorrectly formatted data, preventing incomplete entries from being stored.
- Duplicate entries are minimized through unique identifier checks.

Testing Outcomes

- The module was tested with various data inputs to ensure reliability and robustness.
- Error alerts appeared appropriately for empty or malformed inputs.
- On successful submission, data was stored in the database and instantly reflected on the trainee dashboard.
- The form displayed consistent performance across different screen resolutions.

Result

The WTC form module delivered a stable and responsive user experience, allowing for accurate, real-time trainee data entry and management. It supports the smooth operation of WTC training workflows and maintains data integrity throughout the process.

WTC Candidate Registration

Personal Details

Contact Details

Professional Details

Course Details

Personal Detail

Picture *

Choose File

No file chosen, image must be portrait with height not ...

Name *

Enter name

Gender *

Select option

Father's Name *

Enter father's name

Mother's Name

Enter mother's name

Date of Birth *

dd-mm-yyyy

Category *

Select option

PWD (Yes/No) *

Select option

Nationality *

Indian

Save and Next →

Fig. 5.3 WTC Registration

5.2.3 Non-Railway Form

The Non-Railway Form Module is tailored to register and manage candidates who are not directly affiliated with railway training programs but are enrolled through external institutions or partnerships. This module offers a structured interface for administrators to input and manage relevant trainee data.

Functionality and Validation

- The form captures essential fields such as name, ID, batch, department, and contact information.
- Manual validation ensures required fields are completed and inputs are in the correct format.
- Unique ID checks help avoid duplicate entries and maintain data accuracy.

Testing Outcomes

- The form was tested for both valid and invalid data inputs.
- The system correctly flagged missing or malformed entries, maintaining form integrity.
- Upon successful submission, the data was stored in the SQLite3 database and updated in real-time on the trainee dashboard.
- The form was responsive and functioned consistently across different devices.

Result

The Non-Railway form module functioned effectively, providing a reliable, user-friendly, and accurate interface for managing candidate information. It streamlines data entry for external candidates and integrates seamlessly with the overall training management system

Non-Railway Candidate Registration

Personal Details
 Contact Details
 Professional Details

Personal Details

Picture *

Choose File
No file chosen

Name *

Enter full name

Gender *

Select gender
▼

Father's Name *

Enter father's name

Mother's Name

Enter mother's name

Date of Birth *

dd-mm-yyyy
📅

Category *

Select category
▼

PWD (Yes/No) *

Select
▼

Nationality *

Indian

Save and Next →

Fig. 5.4 Non Railway Registration

5.2.4 Image Handling

The Image Handling Module enables administrators to upload and associate trainee photographs as part of the registration process. This feature ensures that each candidate profile contains a visual identity, improving authenticity and verification during training processes.

Functionality and Validation

- The module allows image upload in standard formats (e.g., JPG, PNG) with file size restrictions to ensure performance.
- It integrates seamlessly into the registration forms for STC, WTC, and Non-Railway candidates.
- Manual validation ensures that a valid image is uploaded and linked to the corresponding trainee ID.

Testing Outcomes

- The system was tested for various scenarios including missing images, unsupported formats, and large file sizes.
- Errors were correctly flagged when invalid or missing image files were submitted.

- Valid images were stored and rendered correctly on the trainee dashboard in real-time.
- The upload feature worked consistently across modern browsers and screen resolutions.

Result

The Image Handling Module performed efficiently, maintaining data integrity and enhancing candidate profiles. It contributed to a more complete and professional training record management experience within the system.

5.2.5 Filtration

The Filtration Module is designed to help administrators efficiently search, filter, and retrieve trainee records based on key attributes such as name, trainee ID, batch, department, or training category (STC, WTC, Non-Railway). This module enhances data accessibility and simplifies administrative tasks related to candidate management.

Functionality and Validation

- The system supports dynamic filtering options that instantly update displayed results based on selected criteria.
- Multiple filters can be applied simultaneously, such as batch and department, to narrow down results.
- Input fields are validated to ensure correct data types are used (e.g., numeric IDs, alphabetic names).

Testing Outcomes

- The module was tested with various combinations of filters and edge cases, including empty fields and invalid queries.
- The system accurately returned relevant trainee records based on the selected parameters.
- Invalid or unmatched queries displayed appropriate “no results found” feedback.
- Filtering speed and responsiveness remained consistent, even with a large number of records.

Result

The Filtration Module proved to be highly effective in managing and navigating trainee records. It offers quick data access, accurate filtering, and supports smooth operation of the dashboard, making administrative workflows more efficient and user-friendly.

The geolocation feature is crucial in discouraging proxy attendance, as it ensures that students are physically present in the designated area when marking their attendance.

5.3 STC MANAGEMENT SYSTEM

5.3.1 Feed Marks

The Feed Marks Module empowers instructors and administrators to enter, update, and manage trainee-specific data such as marks, performance scores, and training evaluations. It plays a crucial role in maintaining accurate academic records and generating standardized output documents like marksheets and performance summaries.

Functionality and Validation

- Enables creation, reading, updating, and deletion (CRUD) of marks data per trainee.
- Marks are entered against specific training sessions or assessments.
- The system includes validations to prevent negative scores, empty fields, or duplicate entries.
- Marksheet format is standardized for consistency across all training categories.

Testing Outcomes

- Simulation tests were conducted to verify proper functioning of all CRUD operations.
- Data rendering was found to be consistent, with real-time updates visible on the trainee dashboard.
- Entries were cross-verified against expected outcomes to ensure accuracy.
- The marksheet generation feature produced correctly formatted documents across multiple test runs.

Result

The Feed Marks Module demonstrated high reliability, accurate calculations, and efficient performance. It successfully supported the evaluation workflow and ensured error-free academic record maintenance for STC, WTC, and Non-Railway candidates.

Functionality and Validation

- Supports full CRUD operations (Create, Read, Update, Delete) for trainee data.
- Displays trainee information in a structured format, including profile image, contact details, training history, and assessment scores.
- Allows updates to marks, attendance, and other dynamic fields from a single dashboard.
- Ensures data integrity with input validations and protected fields.

Testing Outcomes

- The module was tested extensively for data accuracy, rendering performance, and editing capabilities.
- Real-time updates were successfully reflected in the trainee dashboard after edits or additions.
- Data rendering remained consistent across multiple sessions and page reloads.
- Output documents like marksheets and certificates generated from profile data were verified for formatting and accuracy.

Result

The Trainee Profile Module proved to be robust, responsive, and intuitive, offering a unified view of each candidate's training journey. It streamlined profile management and played a key role in enabling accurate tracking and reporting.



Geeta
 Summer Intern
 Non Railway Candidate

Basic Information

S.No: 3003

Gender: Female

Father's Name: Somesh

Mother's Name: N/A

DOB: 2004-12-12

Employee No.: S100001

Category: Non Railway

Nationality: Indian

PWD: No

Joining Date: 2025-07-18

Sparing Date: N/A

Course Information

Ticket No.: S100001

Batch: 2024-2025

Stream: Non Railway

Type: Non Railway

Work Information

Designation: Summer Intern

Unit: Degree Holder

Working Under: Er. Pratham

Course Type: Recommendation

Education

Qualification: Bachelor's Degree

Field of Study: B.Tech - Computer Science

Institution: L U

Grade Type: Percentage

Grade/Score: 85

Contact Information

Phone: 1234567890

Email: geeta@outlook.com

Emergency Contact: 1234567890

Permanent Address: Lucknow

Current Address: Lucknow

Fig. 5.6 Candidate Profile

5.3.3 Line Training

The Line Training Module is designed to track and manage the on-field training sessions of trainees, commonly referred to as "line training." This component allows instructors and administrators to record attendance, progress, scores, and schedules related to hands-on training activities conducted on-site or in workshops.

Functionality and Validation

- Enables CRUD operations for recording training schedules, progress notes, and scores.
- Associates practical training sessions with individual trainee profiles for complete training history.
- Provides structured forms to input session details, locations, and remarks.
- Includes validation checks to avoid duplicate or incomplete entries.

Testing Outcomes

- Simulation tests were conducted to evaluate the module's functionality with different training batches and trainees.
- Data rendering remained accurate and responsive during form updates and dashboard refreshes.

- Line training reports and scores were correctly reflected in trainee profiles and aligned with the marksheet structure.
- Reports generated from this module matched expected results in both content and format.

Result

The Line Training Module performed reliably under various test cases, delivering real-time tracking, accurate data entry, and efficient report generation. It plays an essential role in bridging theoretical knowledge with practical exposure for STC, WTC, and Non-Railway trainees.

The screenshot displays the 'Line Training' interface with the following components:

- Header:** 'Line Training' title and 'Manage line training schedules and assignments' subtitle. A 'Cancel' button and '1 Active Programs' indicator are in the top right.
- Section:** '+ Schedule New Line Training'.
- Select Trainees:** A search bar 'Search trainees by name, ticket number...' and a list of three trainees:
 - Ram Bhadur Rai (SSE00001, SSE - RCNK)
 - Subha singh (ASE00002, ASE - RCNK)
 - Riya Tripathi (ASE00001, ASE - RCNK)
- Activity Centre:** A dropdown menu with 'e.g., JAT Diesel Shed'.
- Start Date:** A date picker with 'dd-mm-yyyy' format.
- End Date:** A date picker with 'dd-mm-yyyy' format.
- Description:** A text area with the placeholder 'Brief description of the training program'.
- Buttons:** 'Cancel' and 'Schedule training' at the bottom right.

Fig. 5.7 Line Training Interface

5.3.4 Marksheet Generation

The Marksheet Generation Module is designed to automate the creation of standardized performance documents based on the trainee's recorded scores and evaluations. This module ensures consistency in formatting and eliminates the need for manual entry and formatting of marksheets.

Functionality and Validation

- Pulls data directly from the trainee profile, feed marks, and training records.
- Generates print-ready PDF documents using integrated tools, maintaining institutional formatting standards.
- Each marksheet includes trainee details, session-wise scores, training type (STC, WTC, or Non-Railway), and evaluation remarks.

- Includes options for preview, download, and re-generation after data updates.

Testing Outcomes

- CRUD operations for underlying data (marks, batch info, etc.) were thoroughly tested and accurately reflected in generated documents.
- Marksheet layout, font consistency, and formatting were validated across multiple training types.
- The output was rendered in real-time upon request, and all data matched backend records.
- System behavior was tested for edge cases like missing data or extremely high/low scores, all of which were handled correctly.

Result

The Marksheet Generation Module proved to be accurate, efficient, and user-friendly, significantly reducing the time and errors associated with manual documentation. It supports reliable and consistent output of trainee performance records.

The screenshot displays the 'Generate Marksheet' interface. At the top, there's a header with a back arrow, the title 'Generate Marksheet', a subtitle 'View and generate trainee marksheets', a 'System Active' status indicator, and a 'Current Date' of '20 July 2023'. Below the header is a 'Search Candidate' section with the instruction 'Find trainee to generate marksheet'. It contains two buttons: 'Search by Ticket Number' (highlighted in orange) and 'Select from Dropdown'. Underneath is a 'Ticket Number' input field with a placeholder 'Enter ticket number (e.g., STC2024001)', a 'Search' button (highlighted in orange), and a 'Reset' button. The main content area below shows a document icon and the text 'No Candidate Selected', followed by a message: 'Please search for a candidate using their ticket number or select from the dropdown to view their marksheet.'

Fig. 5.8 Marksheet Generation Interface

5.4 WTC MANAGEMENT

5.4.1 Trainee Profile

The Trainee Profile Module under WTC Management is designed to streamline and automate the handling of training-related records for Workshop Training Centre (WTC) candidates. It enables administrators to manage individual training details, generate official letters, and issue certificates directly through the system.

Functionality and Validation

- Automatically retrieves WTC trainee information and links it with training sessions.
- Supports generation of official documents like offer letters, completion certificates, and joining memos.
- Utilizes the html2pdf library to convert dynamic HTML content into standardized PDFs that match institutional templates.
- Allows data updates to be reflected instantly in regenerated documents.

Testing Outcomes

- The system accurately displayed all relevant trainee information upon accessing individual profiles.
- Document generation was tested across various scenarios and compared with manually created templates for consistency.
- PDF outputs were visually aligned with expected formatting, containing proper headers, trainee details, and signatures.
- Any updates made to trainee data were successfully propagated to the corresponding documents in real-time.

Result

The Trainee Profile Module under WTC Management functioned smoothly and accurately, offering automated, error-free, and professional-quality document generation. It minimized administrative workload while maintaining the precision and consistency required for official WTC documentation.

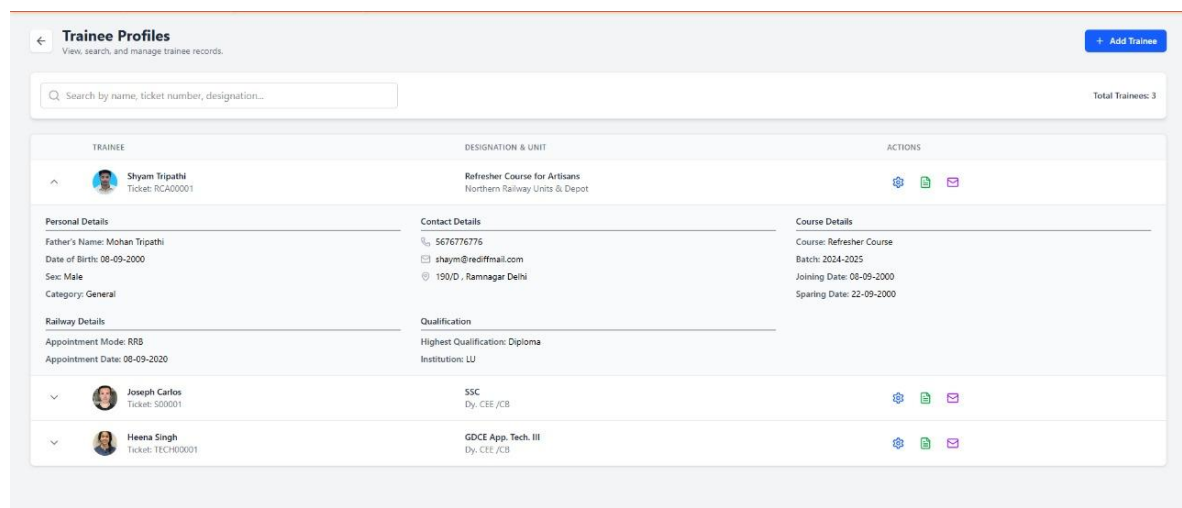


Fig. 5.9 Trainee Profile Page

5.4.2 Attendance System

The Attendance System Module under WTC Management is developed to digitally track and maintain attendance records of Workshop Training Centre (WTC) candidates. It replaces manual registers with a centralized, real-time tracking system that improves accuracy and reduces paperwork.

Functionality and Validation

- Allows administrators to mark daily attendance per candidate and batch.
- Displays cumulative attendance records with timestamps for transparency and traceability.
- Supports filtering by date, batch, or individual ID for quick retrieval and verification.
- Attendance reports are exportable in PDF format using the html2pdf library for official documentation.

Testing Outcomes

- The module was tested across multiple training batches with different attendance patterns.
- All data entries were correctly displayed and stored in the database.
- Modifications or corrections to attendance entries were instantly updated and reflected in the reports.
- The generated PDFs matched manually created templates in structure, content accuracy, and formatting.

Result

The Attendance System module for WTC proved to be reliable, user-friendly, and administratively efficient. It ensured precise tracking of trainee participation and produced standardized attendance reports suitable for internal use and certification purposes.

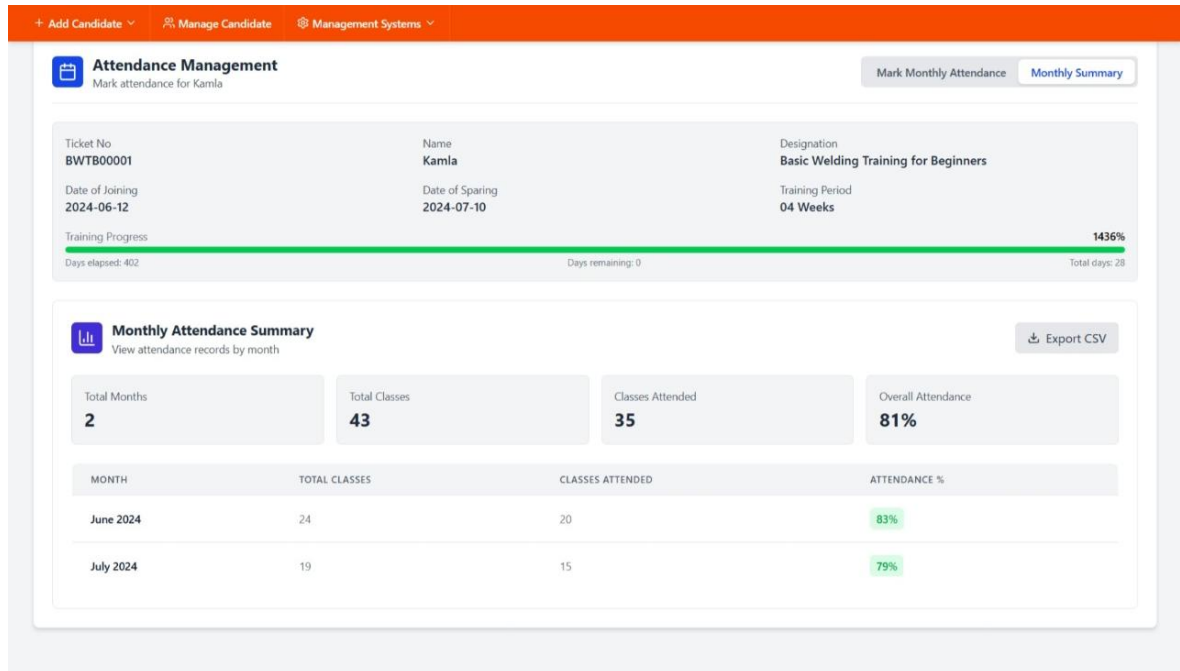


Fig. 5.10 Trainee Attendance Summary

5.4.3 Certificate Generation

The Certificate Generation Module under WTC Management is developed to streamline the issuance of training completion certificates for Workshop Training Centre (WTC) candidates. It automates the document creation process, ensuring accuracy, consistency, and professional formatting aligned with Northern Railway standards.

Functionality and Validation

- Automatically populates certificate templates with trainee details, training duration, course name, and completion status.
- Generates PDF certificates using the html2pdf library with proper formatting, institutional headers, and signature sections.
- Enables preview, re-generation, and printing of certificates directly from the system dashboard.
- Supports instant updates to the certificate if any trainee information or performance data is modified.

Testing Outcomes

- The system successfully pulled data from trainee profiles and marks entries to create complete and accurate certificates.

- PDF outputs were compared with manually designed certificate templates and matched in layout, branding, and structure.
- Edge cases like missing fields or invalid data were correctly flagged before certificate generation.
- Real-time updates ensured data consistency across all generated certificates.

Result

The Certificate Generation Module functioned with high accuracy, fast performance, and professional output quality. It effectively reduced manual efforts while ensuring that WTC trainees received valid, verifiable training completion certificates without delays.

Friday 19 July 2025 Font Size ▾

Northern Railway
Supervisor Training Center
Trainee Management System

Search... Search

Admin Logout

[+ Add Candidate](#) [Manage Candidate](#) [Management Systems ▾](#)

← Certificate Management

Search by name or ticket number... Filter ×

Batch: All Batches ▾ Module: All Modules ▾ Status: All Statuses ▾

☒ 2 of 2 selected Generate Certificates

	Ticket No.	Name	Batch	Module	Training Period	Status
☒	RCW00001	Kishori	25B23		18 Jul 2025 - 8 Aug 2025	Active
☒	CAT100001	Rohan	25A1		18 Jul 2025 - 18 Jul 2026	Active

Fig. 5.11 Certificate Generation Interface

5.4.4 Letter Generation

The Letter Generation Module under WTC Management is built to simplify the creation of various official letters required during the training lifecycle of Workshop Training Centre (WTC) candidates. These may include joining letters, training confirmation letters, or relieving letters.

Functionality and Validation

- Dynamically fetches trainee details such as name, ID, training period, and course information.
- Uses predefined templates to generate official letters in PDF format using the html2pdf library.

- Enables administrators to preview, download, and re-generate letters based on updates to trainee profiles.
- Ensures consistent formatting across all letter types, aligned with institutional standards.

Testing Outcomes

- The module was tested by generating multiple types of letters with varying inputs.
- PDF outputs were visually and structurally consistent with manually issued letters.
- Edits to trainee records were correctly reflected in the regenerated letters. The system validated the presence of all required fields before allowing generation, preventing incomplete outputs.

Result

The Letter Generation Module worked with precision, speed, and ease of use, ensuring official communications for WTC trainees are issued accurately and promptly. It reduces paperwork and supports digital record-keeping with professional-quality documentation.

Friday 19 July 2025 Font Size ▾

Northern Railway
Supervisor Training Center
Trainee Management System

Search... Search

Admin Logout

+ Add Candidate ▾ Manage Candidate Management Systems ▾

Letter Management

Search by name or ticket number... Filter ×

☒ 2 of 2 selected Practical Training ▾ Generate Letters

Ticket No.	Name	Designation	Batch	Module No	Training Period	Status
☒ RCW00001	Kishori	Refresher Course for Welders	25B23		18/07/2025 - 08/08/2025	Active
☒ CAT100001	Rohan	CG Apprentice Technician III	25A1		18/07/2025 - 18/07/2026	Active

Fig. 5.12 Letter Generation Interface

5.5 DISCUSSIONS

The simulation and testing outcomes indicate that the Training Management System performs effectively across all key modules and training domains—STC, WTC, and Non-Railway candidates. Each component of the system, from data entry to document generation, functioned as intended, validating the robustness and stability of the application.

Key Strengths Observed

- **Data Consistency:** Information was accurately maintained across forms, databases, and generated documents.
- **Real-Time Interaction:** Live updates ensured that any changes in data were reflected instantly across modules.
- **Modular Design:** The use of a component-based frontend and MVC backend architecture facilitated organized development, ease of maintenance, and scalability.
- **Document Automation:** Tools like html2pdf and manual validation methods ensured the professional and error-free generation of marksheets, certificates, and letters.
- **User-Centric Interfaces:** Intuitive dashboards and form layouts enhanced usability for administrators and instructors.

Scope for Future Enhancements

- **Biometric Attendance Integration:** To strengthen authenticity and automate trainee verification.
- **Push Notification System:** For updates on training schedules, attendance, and document availability.
- **Advanced Analytics Dashboard:** To track performance trends, generate comparative reports, and assist in decision-making.
- **Mobile Application Support:** Extending functionality to mobile platforms for increased accessibility and field-level data entry.

5.6 CONCLUSION

Overall, the system delivers a functional, responsive, and scalable solution tailored for the digitization of training workflows. The results confirm that it can be confidently deployed within institutional environments while also laying the foundation for future upgrades and integration.

CHAPTER 6

CONCLUSIONS AND SCOPE FOR IMPROVEMENT

6.1 INTRODUCTION

This chapter presents the conclusion of the project titled "Training Management System for STC, WTC, and Non-Railway Candidates", summarizing the development journey, system performance, and future enhancement roadmap.

The system successfully addresses the primary objective of digitizing training management workflows for the Supervisor's Training Centre (STC), Workshop Training Centre (WTC), and Non-Railway training programs at Northern Railway, Charbagh, Lucknow. The platform eliminates manual data handling inefficiencies by automating candidate registration, batch allocation, attendance tracking, examination performance monitoring, and certificate generation.

The system was built using a modern technology stack consisting of React.js with Tailwind CSS for the frontend, Node.js with Express.js for the backend, and SQLite3 for data storage. Additional tools including html2pdf for report generation and Nodemon for development efficiency were integrated to enhance functionality and developer productivity.

6.2 CONCLUSIONS

6.2.1 Objective Achievement

The project successfully delivered a centralized platform that manages the complete training lifecycle for all three candidate categories. The system replaced traditional paper-based processes with a fully automated digital workflow, supporting candidate registration through certificate issuance with minimal manual intervention.

6.2.2 Technical Implementation Success

The chosen technology stack proved effective in creating a responsive, maintainable system. React.js enabled component reusability across modules, while Tailwind CSS facilitated rapid UI development. The Node.js backend efficiently handles routing and business logic, with SQLite3 providing reliable data storage for the current offline deployment requirements.

6.2.3 Functional Module Delivery

The system provides comprehensive functionality including separate workflows for each training category, profile management with image upload capabilities, marks entry and management, attendance tracking, and automated document generation. The modular architecture ensures clean separation of concerns and maintainable code structure.

6.2.4 Performance Impact

Implementation results demonstrate significant operational improvements including reduced manual effort, streamlined workflows, minimized human error, and enhanced data accuracy. The system enables real-time updates and instant report generation, substantially improving administrative efficiency.

6.2.5 System Reliability

Thorough testing across various scenarios confirmed system stability, data accuracy, and responsive performance across different screen sizes. All core modules including form submissions, data storage, filtering, and artifact generation function as designed.

6.2.6 Performance and Efficiency Gains

- The system has significantly reduced manual effort, streamlined workflows, and minimized the chances of error.
- It supports real-time updates, centralized data storage, and instant report generation—enhancing operational speed and accuracy.

- The platform also supports quick retrieval and filtering of candidates for reporting or evaluation.

6.3 FUTURE SCOPE

The current system establishes a solid foundation for digital training management, with several strategic enhancement opportunities identified for future development

6.3.1 Online Deployment and Accessibility

Transition from offline to online deployment to enable remote access and broader institutional reach.

- Cloud-based hosting on platforms such as AWS, Vercel, or Heroku
- Implementation of responsive web design for mobile and tablet access
- Development of Progressive Web App (PWA) capabilities for offline functionality when needed

6.3.2 Database Enhancement and Scalability

Transition from offline to online deployment to enable remote access and broader institutional reach.

- Cloud-based hosting on platforms such as AWS, Vercel, or Heroku
- Development of Progressive Web App (PWA) capabilities for offline functionality when needed

6.3.3 Security Infrastructure and Access Control

Implement comprehensive security measures and granular access management.

- Development of Role-Based Access Control (RBAC) system with defined user permissions
- Implementation of individual user accounts with unique credentials replacing current shared role-based login
- Integration of JWT authentication for secure session management
- Addition of comprehensive audit logs for tracking user activities and system changes

- Implementation of HTTPS encryption and data protection measures

6.3.4 Advanced User Management System

Create sophisticated user role management with personalized access controls.

- Development of individual user profiles with specific role assignments
- Implementation of hierarchical permission structures (Admin, Trainer, Data Entry, Viewer roles)
- Creation of user activity monitoring and access logging capabilities
- Integration of password management and account security features

6.3.5 Enhanced Reporting and Analytics

Develop comprehensive reporting capabilities with advanced data visualization.

- Implementation of customizable report templates with improved formatting options
- Integration of data analytics dashboards for performance tracking and trend analysis
- Development of automated report scheduling and distribution features
- Addition of export capabilities in multiple formats (PDF, Excel, CSV)

6.4 CONCLUSION

The Training Management System represents a successful demonstration of modern web development principles applied to solve real-world administrative challenges. The project effectively combines frontend and backend technologies to create a functional, user-friendly platform that significantly improves training management efficiency.

The system's modular architecture provides a robust foundation for future enhancements, positioning it for potential expansion into a comprehensive Training ERP solution. The identified future scope areas will transform the current system into an enterprise-ready platform capable of supporting large-scale, multi-institutional training operations.

This project validates the effectiveness of thoughtful technology selection and structured development practices in creating practical solutions that deliver measurable improvements to traditional administrative processes. The successful implementation

provides a scalable framework for digital transformation in educational and training environments.

APPENDIX

- GitHub Repo: <https://github.com/inspiredrishabh/STVT>



- Project Demonstration: <https://drive.google.com/file/d/1pwAztCKva23m1nWiH-7i5tZ53S2cMQF4/view?usp=sharing>



CONTRIBUTORS

Rishabh Gupta played a pivotal role in managing the overall development of the project. He was responsible for major backend and frontend tasks, especially in the STC modules, and handled key issues that arose during development. Also set up a robust IP-based local server enabling seamless intra-network access and private deployment simulation. His leadership ensured smooth coordination and resolution of technical challenges.

Key Contributions:

- Implemented IP based local network server setup
- Designed and implemented the initial setup of STC, WTC and Non-Railway routes, controller and model
- Developed the ticket number generation mechanism in STC
- Finalized the design of STC management frontend modules
- Refined controller and model logic for STC and Non-Railway routes
- Acted as the primary point for conflict resolution within the team
- Developed dashboard functionality
- Resolved the marksheet supplementary issue
- Carried out minor fixes throughout development
- Initialized 12 course-related backend models, controllers, and routes
- Initialized STC certificate logic

Awanish Yadav contributed extensively to backend development and infrastructure stability. He ensured that data flow, authentication, and storage mechanisms were robust while also working on early frontend prototypes.

Key Contributions:

- Set up initial authentication mechanisms
- Refined controller and model logic for WTC
- Implemented complete backend and database backup for WTC including attendance system
- Helped in IP based local network server setup
- Built ticket number generation feature for WTC and Non Railway
- Developed session auto-logout feature
- Set up prototype and initial WTC frontend
- Enabled Excel sheet export functionality
- Contributed to minor fixes throughout the project

Pratyush Kumar Singh focused on the backend infrastructure of the academic module. He was responsible for implementing course-related data structures and APIs.

Key Contributions:

- Developed 12 STC course-related backend models, controllers, and routes

Priyanshu Raj worked on the frontend of the application, particularly during the early phases. He contributed to building the UI foundation and supported refinement efforts.

Key Contributions:

- Designed the prototype frontend
- Helped in refining the overall frontend design

Somesh Pratap Singh led the UI design efforts for the management and dashboard sections. He took charge of enhancing user interfaces and refining visual components across modules while also contributing greatly to the testing efforts.

Key Contributions:

- Designed the management page UI
- Carried out complete frontend and UI refinement throughout the project
- Built the dashboard UI
- Refined the marksheet UI
- Refined STC certificate logic
- Participated in testing

Shivam Yadav was responsible for data simulation between frontend and backend and ensured that the backend and frontend communicated seamlessly. He also contributed to early development of the dashboard and marksheet modules.

Key Contributions:

- Participated in dashboard creation
- Designed the initial marksheet interface
- Debugged and set up API connection endpoints for frontend-backend data flow

Venkatesh contributed to the initial design of frontend modules and led the documentation efforts. His involvement was key in testing and producing the final project report.

Key Contributions:

- Designed initial STC management frontend modules
- Led testing of the application
- Compiled and authored the final report
- Fetched course details from the source

Pallavi Tripathi supported both testing and documentation activities. She helped in preparing the initial report and ensured quality by participating in testing rounds.

Key Contributions:

- Contributed to the initial project report
- Participated in testing

Shivansh Kumar provided valuable assistance during the early documentation and testing stages. He contributed to the groundwork for project reporting and verification.

Key Contributions:

- Contributed to the initial project report
- Participated in testing
- Fixed image-related issues in the system

In addition to the listed individual contributions, every team member was actively involved in the overall development process throughout the entire project lifecycle. The development began with a prototype model to explore feasibility and gather early feedback. As the project evolved, the team transitioned to an agile development model, enabling iterative improvements, efficient task allocation, and quick adaptation to changes. This collaborative approach ensured that all members played an integral role beyond their defined responsibilities, contributing to planning, testing, debugging, and refining features across modules.

We extend our sincere gratitude to our mentors — **Mr. Manvendra Singh, Mr. Naveen Jaiswal, and Mr. Alok Dixit** — for their invaluable support and guidance. Their clarity in conveying project requirements and their continuous mentorship were instrumental in steering the team through various stages of development. Their insights and feedback not only enhanced the quality of the final product but also enriched the team's learning experience.